

Sadržaj

Predgovor	5
1 Uvod: šta su strukture podataka	7
1.1 Podaci, operacije i organizacija	7
1.2 Reprezentacija i invarijanta	8
2 Memorijski model potreban za strukture podataka	11
2.1 Adrese i pokazivači	11
2.2 Reference	16
2.3 Statički niz i njegova ograničenja	20
2.4 Dinamička alokacija	23
2.5 Životni vek objekata	27
2.6 Osnovna ideja RAIL	31
3 Dinamički niz	33
3.1 Kapacitet i veličina	34
3.2 Klasa dinamičkog niza	35
3.3 Rast niza i cena dodavanja elemenata	39
3.4 Umetanje i brisanje	43
3.5 Kopiranje niza	45
3.6 Dinamički niz u standardnoj biblioteci	48
4 Povezane liste	51
4.1 Jednostruko povezana lista	51
4.2 Čvor liste	52
4.3 Obilazak liste	53
4.4 Umetanje i brisanje	54
4.5 Pretraga	57
4.6 Sortirana lista i učešljavanje	58
4.7 Poređenje liste i niza	59
4.8 Kružna lista	60
4.9 Dvostruko povezana lista	63
4.10 Liste u standardnoj biblioteci	67
4.11 Kada lista jeste, a kada nije dobar izbor	69
5 Stek	71
5.1 ADT steka	71
5.2 Operacije: <i>push</i> , <i>pop</i> , <i>top</i> , <i>empty</i>	71
5.3 Implementacija pomoću niza ili vektora	71
5.4 Implementacija pomoću liste kao alternativa	71
5.5 Primene	71
5.6 Veza sa <i>std::stack</i>	71

6	Red i dek	73
6.1	ADT reda	73
6.2	Operacije nad redom	73
6.3	Implementacija kružnim baferom	73
6.4	Implementacija pomoću povezane liste	73
6.5	Red pomoću dva steka kao primer amortizovane analize	73
6.6	Primene	73
6.7	Dek kao red sa dva kraja	73
6.8	Osnovna ideja implementacije deka	73
6.9	Veza sa <i>std::queue</i> i <i>std::deque</i>	73
7	Hip i red sa prioritetom	75
7.1	Ideja prioriteta	75
7.2	Binarni hip	75
7.3	Implicitna reprezentacija u nizu	75
7.4	Operacije	75
7.5	Formiranje hipa	75
7.6	Hipsort	75
7.7	Primene reda sa prioritetom	75
7.8	Veza sa <i>std::priority_queue</i>	75
8	Binarna stabla	77
8.1	Rekurzivna struktura stabla	77
8.2	Čvor, koren, list, visina	77
8.3	Obilasci stabla	77
8.4	Rekurzivni algoritmi nad stablima	77
8.5	Memorijska reprezentacija stabla	77
9	Binarna stabla pretrage	79
9.1	Uređenje elemenata	79
9.2	Invarijanta BST-a	79
9.3	Pretraga	79
9.4	Umetanje	79
9.5	Brisanje	79
9.6	Složenost u zavisnosti od visine	79
9.7	Problem degenerisanog stabla	79
10	Balansirana stabla	81
10.1	Motivacija za balansiranje	81
10.2	Visina i složenost	81
10.3	AVL stablo ili crveno-crno stablo kao glavni primer	81
10.4	Rotacije	81
10.5	Umetanje	81
10.6	Brisanje	81
10.7	Veza sa <i>std::map</i> i <i>std::set</i>	81

11	Heš-tabele	83
11.1	Asocijativni pristup podacima	83
11.2	Ključ i vrednost	83
11.3	Heš-funkcija	83
11.4	Kolizije	83
11.5	Odvojeno ulančavanje	83
11.6	Otvoreno adresiranje	83
11.7	Faktor opterećenja	83
11.8	Proširivanje i rehash	83
11.9	Prosečna i najgora složenost	83
11.10	Veza sa <i>std::unordered_map</i> i <i>std::unordered_set</i>	83
12	Dodatne i napredne strukture	85
12.1	Skip-lista	85
12.2	B-stablo (B+ stablo)	85
A	Izbor strukture podataka u praksi	87
A.1	Kada koristiti vektor	87
A.2	Kada koristiti listu	87
A.3	Kada koristiti stek	87
A.4	Kada koristiti red	87
A.5	Kada koristiti hip	87
A.6	Kada koristiti stablo	87
A.7	Kada koristiti heš-tabelu	87
A.8	Primeri	87
B	Kratak pregled C-a	89
B.1	Osnovna sintaksa (C i C++ razlike)	89
B.2	C nizovi	89
B.3	C niske	89
B.4	<i>malloc</i> i <i>free</i>	89
B.5	<i>printf</i> i <i>scanf</i>	89
B.6	C datoteke	89

Predgovor

1

Uvod: šta su strukture podataka

Rešavanje problema programom zahteva izbor algoritama, ali i izbor načina na koji će podaci biti organizovani. Isti podaci mogu se čuvati na različite načine, a svaki od njih utiče na to kako se podaci pretražuju, dodaju, menjaju i uklanjaju. *Struktura podataka* (eng. *data structure*) objedinjuje organizaciju podataka i operacije koje se nad njima izvode. Od njenog izbora zavise vremenski i memorijski trošak tih operacija, a time i mogućnosti i ograničenja programa koji je koristi.

Posmatrajmo kolekciju celih brojeva u koju treba dodavati elemente, iz nje ih uklanjati i proveravati da li se u njoj nalazi zadati broj. Sve tri operacije mogu se ostvariti i kada su brojevi smešteni u neuređen niz i kada se čuvaju u uređenom nizu, ali njihova cena nije ista. Strukturu podataka zato ne bираmo prema tome koje operacije nudi, nego prema ceni operacija koje su za dati problem važne i prema tome koliko se često izvršavaju. *Strukturu podataka, dakle, ne bираmo prema tome koje operacije nudi, nego prema ceni operacija koje su za dati problem važne i prema tome koliko se često izvršavaju.*

Isto razumevanje potrebno je i kada se programski kod generiše pomoću alata, poput *velikih jezičkih modela* (eng. *large language models*). Dobijena implementacija može davati ispravne rezultate, a ipak biti neprikladna za očekivanu količinu podataka ili obrazac upotrebe. Pretraga koja redom proverava svaki element, kakvu zahteva neuređen niz iz prethodnog primera, može biti sasvim zadovoljavajuća nad malom kolekcijom, ali kada se često ponavlja nad velikom, lako odnosi najveći deo vremena izvršavanja programa. I kada se pisanje implementacije prepusti alatu, na programeru ostaje da precizira zahteve, proceni predloženo rešenje i obrazloži izbor strukture. Poznavanje strukture podataka daje osnovu za takvu procenu: omogućava da se razume zašto neko rešenje radi, koliko košta i pod kojim uslovima ga treba zameniti drugim.

1.1 Podaci, operacije i organizacija

Vratimo se kolekciji celih brojeva sa početka glave i opišimo je preciznije. Nad kolekcijom se često postavlja pitanje „Da li se broj x nalazi u kolekciji?“, a ona se tokom rada programa menja: povremeno se dodaju novi elementi ili uklanjaju postojeći. Isti broj može se pojaviti više puta, a redosled kojim su elementi dodavani nije važan. Operacija $dodaj(x)$ dodaje jedno pojavljivanje broja x , operacija $sadrzi(x)$ proverava da li postoji bar jedno njegovo pojavljivanje, a $obrisi(x)$ uklanja jedno pojavljivanje, ako ono postoji; ako traženog broja nema, brisanje ne menja kolekciju.

Jedan način da se takva kolekcija predstavi jeste da se njeni elementi smeste u niz, bez ikakvog zahteva u pogledu poretka. Druga mogućnost je da se elementi takođe smeste u niz, ali da se održavaju u neopadajućem poretku. Slika 1.1 prikazuje obe organizacije za kolekciju koja sadrži brojeve 12, 4, 9, 4, 17 i 6. Broj 4 pojavljuje se dvaput u oba niza: promenjena je organizacija, a ne sadržaj kolekcije.

Razmotrimo najpre proveru da li se broj 9 nalazi u kolekciji. U neuređenom nizu elemente pregledamo redom i zaustavljamo se kada pronađemo traženi broj. Ako tražimo broj 8, kojeg u kolekciji nema, mora se pregledati ceo niz, jer nijedan pregledani element ne govori da li se 8 nalazi na nekom od preostalih mesta. Broj proverava je zato u najgorem slučaju srazmeran broju elemenata kolekcije.

neuređen niz	12	4	9	4	17	6
	0	1	2	3	4	5
uređen niz	4	4	6	9	12	17
	0	1	2	3	4	5

Slika 1.1: Ista kolekcija celih brojeva predstavljena neuređenim i uređenim nizom.

U uređenom nizu može se primeniti binarna pretraga. Ako element na sredini posmatranog dela niza nije jednak traženom broju, poređenje sa njim omogućava da se odbaci približno polovina preostalih kandidata, i postupak se ponavlja dok se broj ne pronađe ili dok se ne utvrdi da ga nema. Uređenost, dakle, omogućava da se odgovor dobije bez provere svih elemenata. U neuređenom nizu isti postupak ne bi bio ispravan: poređenje sa jednim elementom ne govori ništa o vrednostima levo i desno od njega.

Prednost pri pretrazi plaća se pri izmenama kolekcije. Da bi se u neuređen niz dodao broj 8, dovoljno je upisati ga na prvo slobodno mesto iza postojećih elemenata, pod pretpostavkom da takvo mesto postoji. U uređenom nizu novi element mora da stane između brojeva 6 i 9, pa se za njega pravi mesto tako što se elementi 9, 12 i 17 pomeraju za po jedno mesto udesno. Binarna pretraga može brzo da odredi mesto umetanja, ali ne uklanja potrebu za pomeranjem. Kako se obezbeđuje dodatni prostor kada je niz popunjen, razmatramo u glavi 3.

Razlika postoji i pri brisanju. Kada se u neuređenom nizu pronađe element koji treba obrisati, njegovo mesto može se popuniti poslednjim elementom niza, posle čega se broj elemenata smanjuje za jedan. Takva promena poretka je dozvoljena jer redosled ne utiče na značenje kolekcije. U uređenom nizu isti postupak mogao bi da naruši uređenost, pa se prazno mesto popunjava pomeranjem narednih elemenata ulevo. Korisno je zato razlikovati dva dela operacije: pronalaženje elementa i uklanjanje pronađenog elementa. Uređenost ubrzava prvi deo, a drugi čak otežava.

Oba niza, prema tome, predstavljaju istu kolekciju i nude iste operacije, ali po različitoj ceni. Kada su upiti brojni, a izmene retke, brža pretraga može opravdati pomeranja pri dodavanju i brisanju; kada su dodavanja česta, a pretrage retke, održavanje uređenosti postaje rad koji se ne isplati. Pri tome se značenje operacija ni u jednom slučaju nije menjalo: *obrisi(x)* u oba niza uklanja jedno pojavljivanje broja x , a razlikuje se samo način na koji se to postiže. Ova dva pogleda na strukturu – šta njene operacije rade i kako su ostvarene – se objašnjava u naredna dva odeljka.

1.2 Reprezentacija i invarijanta

Da bi implementirali apstraktni tip podatka, potrebno je odrediti kako se njegove vrednosti predstavljaju u memoriji. *Reprezentacija* (eng. *representation*) obuhvata konkretan raspored podataka i pravila prema kojima se taj raspored tumači i održava. Pri predstavljanju kolekcije nizom, na primer, potrebno je znati gde su elementi smešteni, koliko ih ima i koji deo raspoloživog prostora zauzimaju, a kod uređenog niza zahteva se još i da elementi budu u neopadajućem poretku.

U dosadašnjem primeru elementi zauzimaju uzastopna mesta u memoriji. Druga mogućnost je da se elementi čuvaju u zasebnim čvorovima, koji ne moraju biti smešteni jedan uz drugi, a da se veze među njima predstavljaju pokazivačima. Memorijske osnove potrebne za razumevanje oba rasporeda razmatramo u glavi 2, a povezane strukture se uvode u glavi 4. Bez obzira na raspored, potrebno je odrediti uslove pod kojima reprezentacija opisuje ispravno stanje strukture.

Ti uslovi čine *invarijantu* strukture (eng. *invariant*). *Invarijanta mora da važi od trenutka kada je objekat stvoren i između svake dve operacije nad njim*. Svaka operacija sme da se osloni na to da invarijanta važi kada operacija počinje, a zauzvrat je dužna da je očuva pravila nakon što se završi.

Ideja je srodna poznatom pojmu invarijante petlje. Razlika je u tome što se uslov sada vezuje za objekat koji postoji tokom izvršavanja programa i nad kojim se operacije mogu pozivati proizvoljnim redosledom.

	Invarijanta petlje	Invarijanta strukture
Kada važi?	Pri svakoj proveru uslova petlje.	Od stvaranja objekta i između svake dve operacije, tokom celog njegovog životnog veka.
Šta je uspostavlja?	Inicijalizacija pre petlje.	Inicijalizacija objekta, odnosno konstruktor.
Šta je održava?	Telo petlje, od jedne provere uslova do sledeće.	Svaka operacija nad strukturom, od ulaska do izlaska.
Kada može privremeno da ne važi?	Tokom izvršavanja tela petlje.	Tokom izvršavanja operacije, dok se reprezentacija menja.
Kako se obezbeđuje?	Pravilnim izborom koraka algoritma.	Pravilnom implementacijom operacija i ograničavanjem neposrednog pristupa reprezentaciji.

Invarijanta strukture je, u tom smislu, trajna osobina objekta: njeno važenje ne zavisi od toga koju će dozvoljenu operaciju korisnik sledeću pozvati. Tokom same operacije ona ipak može privremeno da ne važi. Pri umetanju u uređen niz, na primer, elementi se pomeraju jedan po jedan, i dok pomeranje traje, niz ne predstavlja ni staru ni novu kolekciju. Takvo stanje ostaje unutrašnja stvar operacije; *korisnik ponovo dobija pristup strukturi tek kada je invarijanta uspostavljena*.

Za uređen niz najvažniji deo invarijante odnosi se na poredak elemenata.

Invarijanta uređenog niza. Elementi kolekcije smešteni su u neopadajućem poretku: svaki element, osim poslednjeg, manji je od sledećeg ili mu je jednak.

Jednakost je dozvoljena jer kolekcija može sadržati više pojavljivanja istog broja. Uslov važi i za prazan niz i za niz sa jednim elementom.

Kod binarne pretrage uređenost je *preduslov*, koji mora da obezbedi onaj ko pretragu poziva. Kod strukture zasnovane na uređenom nizu ista osobina postaje invarijanta, za čije je održavanje odgovorna sama struktura. Korisnik zato ne mora da uređuje niz pre svakog poziva *sadrzi(x)*; umesto toga, operacije *dodaj(x)* i *obrisi(x)* moraju biti implementirane tako da uređenost ostane očuvana. Dodavanje smešta novi element na odgovarajuće mesto, a brisanje zatvara nastalu prazninu bez promene međusobnog poretka preostalih elemenata.

Održavanje invarijante istovremeno donosi korist i zahteva rad. Uređenost omogućava da se operacija *sadrzi* ostvari binarnom pretragom u vremenu $O(\log n)$, gde je n broj elemenata kolekcije. Da bi uređenost ostala očuvana posle dodavanja, međutim, ponekad treba pomeriti sve postojeće elemente, pa je složenost operacije *dodaj* u najgorem slučaju $O(n)$. I korist i cena posledica su iste odluke o reprezentaciji.

Neuređen niz nema zahtev u pogledu poretka, ali to ne znači da nema invarijantu: i dalje mora biti ispravno zabeležen broj elemenata i mora postojati prostor u kome su oni smešteni. Odsustvo uslova uređenosti ostavlja, međutim, veću slobodu pri izmenama. Upravo zato je brisanje zamenom poslednjim elementom, opisano u odeljku 1.1, dozvoljeno: apstraktni tip ne pridaje značenje redosledu, a reprezentacija ne postavlja uslov koji bi takvu izmenu zabranio.

Da bi struktura mogla da garantuje svoju invarijantu, korisniku se ne sme dopustiti da proizvoljno menja njenu unutrašnju reprezentaciju. Kada bi korisnik mogao neposredno da promeni neki element uređenog niza, mogao bi da naruši poredak, posle čega binarna pretraga više ne bi davala pouzdan odgovor. Skrivena reprezentacija zato obezbeđuje da se sve izmene obavljaju kroz operacije koje invarijantu održavaju. *Korisnik se oslanja na značenje tih operacija, a njihova implementacija na uslove koji važe u svakom ispravnom stanju strukture.*

Cene operacija u ovoj glavi, kao i u ostatku knjige, izražavamo poznatom notacijom $O(n)$, pri čemu n označava broj elemenata u strukturi. Podsetimo se: za nenegativne funkcije f i g kažemo da je $f(n) = O(g(n))$ ako postoje konstante $c > 0$ i n_0 takve da važi

$$f(n) \leq c \cdot g(n) \quad \text{za svako } n \geq n_0.$$

Definicija daje samo gornju granicu, pa bi za pretragu neuređenog niza formalno bilo tačno i $O(n^2)$; mi, međutim, uve se navodi najmanja granica koja važi, u ovom slučaju $O(n)$.

Ako nije drugačije naglašeno, navodimo vremensku složenost u najgorem slučaju. Kod heš-tabela razmatramo i prosečan slučaj, uz pretpostavke o tome kako heš-funkcija raspoređuje ključeve (glava ??). Amortizovanu ocenu, koja opisuje cenu operacije posmatranu kroz niz uzastopnih operacija, uvodimo u glavi 3, na primeru proširivanja dinamičkog niza.

Pored vremena, razmatramo i prostornu složenost: ukupno zauzeće memorije i dodatni prostor koji zahteva sama organizacija podataka, kao što su pokazivači uz elemente povezane liste ili neiskorišćeni kapacitet dinamičkog niza. Isti red vremenske složenosti, najzad, ne znači i istu brzinu izvršavanja: na nju utiču i raspored podataka u memoriji i način na koji se koristi keš memorija, o čemu je reč u glavi 4.

2

Memorijski model potreban za strukture podataka

Do sada smo podacima pristupali preko imena promenljivih i preko indeksa elemenata niza. Za mnoge algoritme to je dovoljno, ali strukture podataka koje će biti predstavljene u narednim poglavljima zahtevaju još nešto: mogućnost da se zapamti gde se neki objekat nalazi u memoriji i da mu se kasnije pristupi posredno, preko tog zapamćenog mesta. Element povezane liste, na primer, mora da „zna“ gde se nalazi sledeći element, iako ta dva elementa ne moraju biti smeštena jedan pored drugog. Veze te vrste u jeziku C++ izražavaju se *pokazivačima* (eng. *pointer*).

Pokazivači omogućavaju da više delova programa pristupa istom objektu, da se objekti međusobno povezuju i da se radi sa memorijom koja se zauzima tokom izvršavanja programa. Da bi se te mogućnosti koristile ispravno, potrebno je *jasno razlikovati objekat, njegovu vrednost i njegovu adresu*. Pogrešna upotreba pokazivača vodi do pristupa memoriji kojoj program ne sme da pristupa, pa uz svaku operaciju navodimo i uslove pod kojima je dozvoljena.

2.1 Adrese i pokazivači

2.1.1 Memorija i adrese

Program tokom izvršavanja čuva svoje podatke u radnoj memoriji računara. Najmanja jedinica memorije kojoj se može pojedinačno pristupiti jeste *bajt* (eng. *byte*), pa se memorija može zamisliti kao dugačak niz bajtova poređanih jedan za drugim. Da bi se znalo gde se neki podatak nalazi, bajtovi su numerisani: svaki bajt ima svoj redni broj, koji nazivamo njegovom *adresom* (eng. *address*). Adresa ima ulogu sličnu kućnom broju u ulici. Ona ne govori ništa o tome šta se na datom mestu nalazi, ali to mesto jednoznačno određuje.

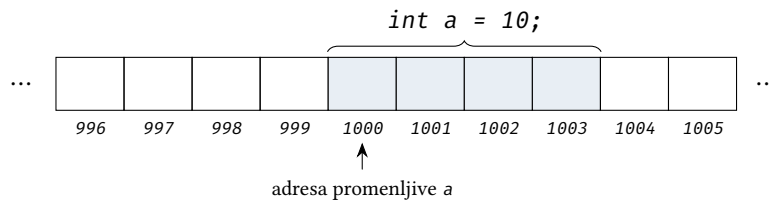
Adrese na kojima će se naći podaci ne bira sam program. Pri pokretanju programa operativni sistem mu dodeljuje deo memorije i stara se da program ne pristupa memoriji koja mu ne pripada. Unutar tog dela memorije prevodilac i izvršno okruženje određuju gde će se smestiti svaka promenljiva. Program, međutim, može da sazna adresu svake svoje promenljive i da preko te adrese promenljivoj pristupi. Na toj mogućnosti zasniva se sve što sledi u ovom odeljku.

Jedan bajt može da sadrži samo 256 različitih vrednosti, pa većina podataka zauzima više uzastopnih bajtova. Koliko bajtova zauzima promenljiva, određuje njen tip. Na većini savremenih sistema tip *int* zauzima četiri bajta, pa se deklaracijom

```
int a = 10;
```

za promenljivu *a* rezervišu četiri uzastopna bajta, a vrednost 10 se zapisuje u njih. Ako prvi od tih bajtova ima adresu 1000, promenljiva zauzima bajtove sa adresama 1000, 1001, 1002 i 1003. Adresom promenljive smatra se adresa njenog prvog bajta, pa je adresa promenljive *a* jednaka 1000. Slika 2.1 prikazuje ovaj raspored.

Broj bajtova koje zauzima tip ili objekat daje operator *sizeof*. Ako je *a* promenljiva tipa *int*, izrazi *sizeof(int)* i *sizeof(a)* imaju istu vrednost. Jezik propisuje da tip *char* zauzima tačno jedan



Slika 2.1: Memorija kao niz bajtova. Svaki bajt ima adresu; promenljiva `a` tipa `int` zauzima četiri uzastopna bajta, a njena adresa je adresa prvog od njih. Adrese su ilustrativne.

bajt, dok za ostale tipove propisuje samo najmanji opseg vrednosti koji moraju da podrže. Stvarne veličine zavise od prevodioca i arhitekture računara, pa se uz primere sa konkretnim adresama uvek navodi koliko bajtova zauzimaju tipovi koji se u njima javljaju.

Koliko adresa postoji, zavisi od toga koliko bitova arhitektura računara koristi za zapis adrese. Na 32-bitnom sistemu adresa je 32-bitni broj, pa program može da adresira najviše 2^{32} bajtova, odnosno 4 GiB memorije. Na 64-bitnom sistemu adrese su 64-bitne, pa je broj mogućih adresa 2^{64} (što daleko prevazilazi količinu memorije bilo kog današnjeg računara). Stvarne adrese su zato veliki brojevi i obično se ispisuju u heksadekadnom zapisu, bez vodećih nula, na primer `0x7ffc3a9e1b2c`. Na slici i u primerima umesto njih koristimo male decimalne brojeve poput 1000. Takvi brojevi se lakše čitaju, a čuvaju ono što je bitno: susedni bajtovi imaju uzastopne adrese. Konkretna vrednost adrese se može razlikovati od jednog izvršavanja programa do drugog, pa se ispravan program nikada ne oslanja na nju.

Za svaku promenljivu treba razlikovati njenu *vrednost* i njenu *adresu*. Vrednost je sadržaj zapisan u bajtovima koje promenljiva zauzima, a adresa je mesto na kome se ti bajtovi nalaze. Naredba `a = 5;` menja sadržaj tih bajtova, ali promenljiva ostaje na istom mestu. *Adresa promenljive se ne menja sve dok promenljiva postoji.* Adresa je, prema tome, i sama podatak: broj koji se može zapamtiti i kasnije upotrebiti. Promenljive koje čuvaju takve podatke uvodimo u nastavku.

2.1.2 Pokazivači, adresni operator i dereferenciranje

Pokazivač je promenljiva čija je vrednost adresa nekog objekta. Kada pokazivač sadrži adresu objekta, kažemo da *pokazuje* na taj objekat. Svaki pokazivač ima tip koji određuje tip objekta na koji pokazuje: pokazivač na `int` ima tip `int*`, pokazivač na `double` tip `double*`, a na isti način se grade i pokazivači na korisnički definisane tipove.

U deklaraciji pokazivača koristi se zvezdica:

```
int *p1;
int* p2;
int* p3, p4, *p5;
```

Razmaci ne utiču na značenje, pa su `p1`, `p2`, `p3` i `p5` pokazivači na `int`. Promenljiva `p4`, međutim, jeste obična celobrojna promenljiva: zvezdica se odnosi samo na promenljivu uz koju stoji, a ne na sve promenljive u deklaraciji. Da bismo izbegli tu zamku, u nastavku svaki pokazivač deklarišemo zasebno, a zvezdicu pišemo uz tip.

Adresu objekta daje unarni operator `&`, koji nazivamo *adresni operator*:

```
int a = 10;
int* p = &a;
```

Izraz $\&a$ daje adresu promenljive a , i ta adresa postaje vrednost pokazivača p ; od tog trenutka p pokazuje na a . Posle ove deklaracije postoje dva različita objekta: promenljiva a sa vrednošću 10 i pokazivač p čija je vrednost adresa promenljive a . Pokazivač je i sam objekat, pa i on zauzima memoriju i ima sopstvenu adresu. Ako a počinje na adresi 1000, a p na adresi 2000, vrednost pokazivača p jeste 1000, dok izraz $\&p$ daje 2000.

Veličina pokazivača ne zavisi od veličine objekta na koji pokazuje. Na većini savremenih 64-bitnih sistema pokazivač zauzima osam bajtova, ali jezik to ne propisuje; stvarnu veličinu daje izraz `sizeof(p)`.

Pristup objektu preko pokazivača ostvaruje se unarnim operatorom `*`. Ta operacija naziva se *dereferenciranje* (eng. *dereferencing*): ako p pokazuje na a , izraz $*p$ označava sam objekat a , a ne njegovu kopiju.

```
int a = 10;
int* p = &a;

int b = *p;
*p = 5;
```

Pri inicijalizaciji promenljive b čita se vrednost objekta na koji p pokazuje, pa b dobija vrednost 10. Naredba `*p = 5;` zatim menja upravo taj objekat: a dobija vrednost 5, dok b zadržava vrednost 10, jer je dobila kopiju ranije vrednosti. Dereferencirani pokazivač može se pojaviti i u složenijem izrazu:

```
*p = *p + a + 3;
```

Pošto p pokazuje na a , oba izraza $*p$ i a na desnoj strani imaju vrednost 5, pa se u a upisuje 13.

Izraz $*p$ sme da stoji na levoj strani dodele jer označava objekat čija se vrednost može menjati; takav izraz nazivamo *l-vrednost* (eng. *lvalue*). Ime a i izraz $*p$ tako predstavljaju dva načina pristupa istom objektu, a tu pojavu nazivamo *alijasovanje* (eng. *aliasing*). Treba pri tome razlikovati dve uloge zvezdice: u deklaraciji `int* p` ona kaže da je p pokazivač, a u izrazu $*p$ ona je operator dereferenciranja. Operatori $\&$ i $*$ deluju u suprotnim smerovima: $\&a$ od objekta daje adresu, a $*p$ od adrese daje objekat.

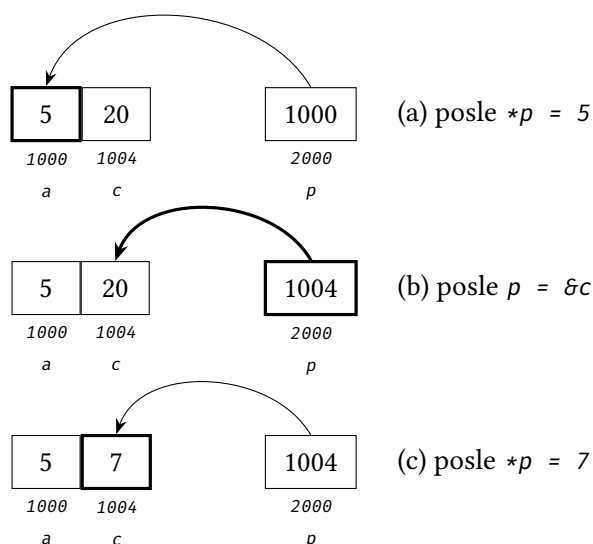
Posebno je važno razlikovati promenu pokazivača od promene objekta na koji on pokazuje:

```
int a = 10;
int c = 20;
int* p = &a;

*p = 5;    // menja se a
p = &c;    // menja se p; a i c ostaju isti
*p = 7;    // menja se c
```

Dodela *preko* pokazivača menja sadržaj objekta na koji pokazivač trenutno pokazuje, dok dodela *pokazivaču* menja samo to na koji objekat on pokazuje. Slika 2.2 prikazuje stanje memorije posle svake od tri dodele.

Pokazivačka vrednost se, kao i svaka druga, može kopirati:



Slika 2.2: Promena objekta i promena pokazivača. (a) Dodela $*p = 5$ menja objekat a ; (b) dodela $p = \&c$ menja samo vrednost pokazivača; (c) dodela $*p = 7$ menja objekat c . Podebljan je deo memorije koji se u datom koraku menja. Adrese su ilustrativne, uz pretpostavku da *int* zauzima četiri bajta.

```
int a = 10;
int* p = &a;
int* q = p;

*q = 20;
```

Inicijalizacijom pokazivača q kopira se adresa sačuvana u p , pa oba pokazivača pokazuju na isti objekat i posle naredbe $*q = 20$; izrazi a , $*p$ i $*q$ imaju vrednost 20. Kopiranje pokazivača, dakle, ne stvara kopiju objekta. Sami pokazivači su i dalje zasebne promenljive: kasnija promena vrednosti pokazivača q ne menja p . Na razliku između kopiranja pokazivača i kopiranja objekta vrat ćemo se kod plitkog kopiranja, u odeljku 2.5.

2.1.3 Multi pokazivač

Iako adrese na slikama zapisujemo brojevima, pokazivački tipovi nisu celobrojni. Jezik zato ne dozvoljava da se pokazivaču dodeli ceo broj, niti celobrojnoj promenljivoj pokazivač:

```
int a = 10;
int* pa = &a;

pa = a; // greška: int se ne pretvara implicitno u int*
a = pa; // greška: int* se ne pretvara implicitno u int
pa = 1234; // greška: 1234 nije pokazivačka vrednost
```

Time se sprečava slučajno mešanje podataka i adresa. Eksplicitne konverzije između celobrojnih i pokazivačkih tipova postoje, ali nam za implementaciju struktura podataka nisu potrebne.

Jedna pokazivačka vrednost ipak ima posebnu ulogu. Pokazivač koji ne pokazuje ni na jedan objekat dobija vrednost *null ptr* i nazivamo ga *multi pokazivač* (eng. *null pointer*):

```
int* p = nullptr;
```

Nultim pokazivačem se označava odsustvo objekta, na primer element povezane strukture koji nema sledbenika ili pretraga koja nije pronašla traženi objekat. Nulti pokazivač nije isto što i pokazivač na objekat čija je vrednost nula:

```
int a = 0;
int* p = &a;      // p pokazuje na postojeći objekat vrednosti 0
int* q = nullptr; // q ne pokazuje ni na jedan objekat
```

Izraz `*p` ovde daje vrednost 0, dok se `q` ne sme dereferencirati. Pre pristupa objektu zato proveravamo da pokazivač nije nulti:

```
if (p != nullptr) {
    *p = 5;
}
```

Isti uslov može se zapisati i kraće, kao `if (p)`, ali u ovoj knjizi pišemo eksplicitno poređenje sa `nullptr`, da bi smisao provere bio neposredno vidljiv. U starijem kodu za nulti pokazivač se sreću i zapisi `0` i `NULL`; savremena preporuka je da se koristi isključivo `nullptr`.

Lokalni pokazivač deklarisan bez inicijalizacije, kao u zapisu `int* p;`, ne dobija automatski vrednost `nullptr`. Njegova vrednost je neodređena i pre upotrebe mu se mora dodeliti ispravna pokazivačka vrednost. Zato pokazivač inicijalizujemo odmah pri deklaraciji: adresom postojećeg objekta ili, ako takvog objekta još nema, vrednošću `nullptr`.

Dereferenciranje nultog pokazivača dovodi do *nedefinisanog ponašanja* (eng. *undefined behavior*): jezik ne propisuje šta se tada dešava. Program se može prekinuti, ali se ne sme pretpostaviti da će svaka pogrešna upotreba pokazivača biti otkrivena na taj način. Ni provera `p != nullptr` nije potpuna provera ispravnosti pokazivača: objekat na koji on pokazuje mora i dalje da postoji, o čemu je reč u odeljku 2.5.

2.1.4 Pokazivači na strukture

Pokazivač može pokazivati i na objekat strukture. Posmatrajmo tip kojim predstavljamo razlomak:

```
struct Razlomak {
    int brojilac;
    int imenilac;
};

Razlomak a{3, 4};
Razlomak* pa = &a;
```

Promenljiva `a` predstavlja razlomak $3/4$, a pokazivač `pa` sadrži adresu tog objekta. Članu `imenilac` neposredno pristupamo izrazom `a.imenilac`. Preko pokazivača najpre dereferenciramo pokazivač, a zatim pristupamo članu dobijenog objekta, izrazom `(*pa).imenilac`. Zagrade su neophodne jer operator `.` ima viši prioritet od unarnog operatora `*`: bez njih bi se izraz `*pa.imenilac` tumačio kao `*(pa.imenilac)`, što nije ispravno, jer `pa` nije objekat strukture nego pokazivač.

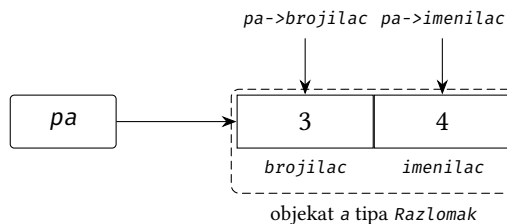
Pristup članu preko pokazivača toliko je čest da za njega postoji poseban operator `->`: izrazi `(*pa).imenilac` i `pa->imenilac` imaju isto značenje. Preko operatora `->` član se može i čitati i menjati:

```
std::cout << pa->brojilac << '/' << pa->imenilac << '\n'; // 3/4

pa->brojilac = 5;
pa->imenilac = 8;

std::cout << a.brojilac << '/' << a.imenilac << '\n'; // 5/8
```

Dodele preko pokazivača *pa* menjaju članove izvornog objekta *a*; novi razlomak ne nastaje. Slika 2.3 prikazuje stanje pre tih dodela: pokazivač čuva adresu celog objekta, a operator *->* bira jedan njegov član.



Slika 2.3: Pokazivač *pa* čuva adresu objekta *a* tipa *Razlomak*. Izrazi sa operatorom *->* pristupaju pojedinačnim članovima tog objekta.

Za operator *->* važe isti uslovi kao za dereferenciranje: pokazivač mora pokazivati na postojeći objekat odgovarajućeg tipa. Ako je *pa* nulti pokazivač, izraz *pa->brojilac* dovodi do nedefinisane ponašanja, iako operator *** u njemu nije eksplicitno zapisan.

Pokazivači na strukture čine osnovu povezanih struktura podataka. Čvor povezane liste u glavi 4 biće struktura koja, pored podatka, sadrži i pokazivač na sledeći čvor, pa logički redosled elemenata više neće zavisiti od toga gde se oni nalaze u memoriji.

2.2 Reference

Pored pokazivača, C++ ima još jedan način da se postojećem objektu pristupi pod drugim imenom. Referenca tipa *T&* vezuje se za objekat tipa *T* i od tog trenutka je njegov alijas:

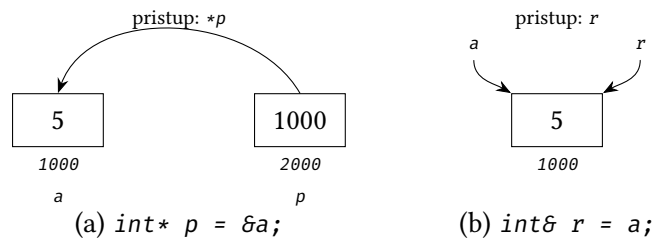
```
int a = 10;
int& r = a;

r = 5; // menja se a
a = 12; // citanjem r sada se dobija 12
```

Deklaracijom *int& r = a*; ne nastaje novi celobrojni objekat. Oba imena označavaju isti objekat, pa i izraz *&r* daje istu adresu kao *&a*. Referenca se mora inicijalizovati pri deklaraciji, a jednom uspostavljena veza ne može se raskinuti: posle *r = b*; referenca *r* i dalje označava *a*, a u *a* je prepisana vrednost promenljive *b*.

Time su već date i osnovne razlike u odnosu na pokazivač, koje slika 2.4 prikazuje u memoriji. Pokazivač je zaseban objekat sa sopstvenom adresom: njegova vrednost je adresa drugog objekta i može se menjati, a pristup objektu zahteva dereferenciranje. Referenca nije zaseban objekat kome bi se mogla uzeti adresa, nego drugo ime postojećeg objekta.

Razlike se pregledno sažimaju u sledećoj tabeli.



Slika 2.4: Pokazivač i referenca na isti objekat. (a) Pokazivač p je zaseban objekat, smešten na sopstvenoj adresi, a objektu a pristupa se izrazom $*p$. (b) Referenca r nije zaseban objekat kome se može uzeti adresa: a i r su dva imena istog objekta, pa i $&r$ daje adresu 1000.

Svojstvo	Pokazivač	Referenca
inicijalizacija pri deklaraciji	nije obavezna	obavezna
veza sa objektom	može se preusmeriti	stalna
stanje „nema objekta”	<code>nullptr</code>	ne postoji
pristup objektu	<code>*p</code> , <code>p->clan</code>	<code>r</code> , <code>r.clan</code>
sopstvena adresa	ima je	nema je

2.2.1 Prenos argumenata

Razlika između kopiranja objekta i pristupa postojećem objektu najčešće se sreće pri pozivanju funkcija. Ako se parametar prenosi po vrednosti, funkcija radi sa kopijom, pa izmene ne dopiru do argumenta; referentni parametar tu kopiju ne pravi, nego se vezuje za sam argument. Uz referentne parametre, razmena vrednosti dve promenljive zapisuje se ovako:

```
void razmeni(int& a, int& b)
{
    int tmp = a;
    a = b;
    b = tmp;
}
```

Isti posao može da obavi i funkcija kojoj se umesto samih promenljivih proslede njihove adrese:

```
void razmeni(int* pa, int* pb)
{
    int tmp = *pa;
    *pa = *pb;
    *pb = tmp;
}
```

Ova verzija poziva se kao `razmeni(&x, &y);`. Parametri pa i pb prenose se po vrednosti, kao i svaki drugi parametar, ali je ono što se kopira adresa, a ne sam objekat. Kopija adrese vodi do istog objekta kao original, pa dodele preko $*pa$ i $*pb$ menjaju upravo x i y . Prenos po referenci i prenos adrese zato postižu isti cilj na dva načina: referenca objekat čini dostupnim neposredno, a pokazivač preko njegove adrese.

Razlika između dve verzije vidi se na mestu poziva. Poziv `razmeni(x, y);` ne odaje da funkcija menja svoje argumente, dok se u pozivu `razmeni(&x, &y);` adresni operator vidi, pa je namera

funkcije očigledna i bez gledanja u njenu deklaraciju. Uz to, funkcija sa referentnim parametrima uvek dobija postojeće objekte, dok funkcija sa pokazivačima mora da računa i na vrednost *nullptr*. Ako pozivalac sme da prosledi *nullptr*, funkcija taj slučaj proverava; ako ne sme, taj uslov se navodi uz opis funkcije.

2.2.2 Referenca na konstantan objekat

Referentni parametar ne služi samo izmeni argumenta. Često je potrebno da funkcija pročita postojeći objekat, a da ga pri tome ne kopira. Za to se koristi tip *const T&*, odnosno referenca preko koje se objekat posmatra kao konstantan:

```
void ispisi(const std::string& s)
{
    std::cout << s << '\n';
}
```

Pri pozivu *ispisi(poruka);* parametar *s* vezuje se za postojeći string, bez njegovog kopiranja. Funkcija preko *s* može da čita string, ali ne i da ga menja: naredba *s.clear();* bila bi greška pri prevođenju. Ključna reč *const* ograničava, dakle, pristup preko date reference, a ne čini sam objekat konstantnim:

```
int a = 5;
const int& r = a;

// r = 6;    // greska: izmena preko r nije dozvoljena
a = 6;       // dozvoljeno; citanjem r sada se dobija 6
```

Obrnuto ne važi: ako je sam objekat definisan kao konstantan, za njega se ne može vezati referenca *int&*, niti se njegova adresa može upisati u pokazivač *int**, jer bi se tako zaobišla zaštita koju *const* daje. Dozvoljeni su samo *const int&* i *const int**.

Izbegavanje kopiranja nije samo po sebi razlog da svaki parametar postane referenca. Za tipove poput *int* i *double* kopiranje je jeftino, pa je prenos po vrednosti prirodan izbor kada funkcija treba samo da pročita argument. Tip *const T&* koristi se za objekte čije kopiranje može biti skupo, kakvi su stringovi, kontejneri i strukture podataka koje gradimo.

2.2.3 Referenca kao povratna vrednost

Funkcija može i da vrati referencu na postojeći objekat. Rezultat poziva tada omogućava pristup tom objektu, umesto da bude nezavisna kopija njegove vrednosti. Posmatrajmo funkciju koja vraća referencu na element niza:

```
int& element(int* podaci, int indeks)
{
    return podaci[indeks];
}
```

Pretpostavlja se da *podaci* ukazuje na početak postojećeg niza i da je *indeks* u njegovim granicama. Pošto je povratni tip *int&*, poziv *element(niz, 1)* označava sam element *niz[1]* i sme da stoji na levoj strani dodele:

```
int niz[] = {10, 20, 30};

element(niz, 1) = 99;      // menja se niz[1]

int kopija = element(niz, 0);
kopija = 0;                // niz[0] ostaje 10
```

Inicijalizacijom promenljive *kopija* čita se, nasuprot tome, vrednost vraćenog objekta i prepisuje u novu promenljivu.

Na tom mehanizmu počiva operator `[]` dinamičkog niza u odeljku 3.2. Da bi izraz oblika `a[i] = 99;` menjao element strukture, operator indeksiranja mora da omogući pristup samom elementu, što se postiže povratnim tipom `T&`; za pristup koji dozvoljava samo čitanje koristi se povratni tip `const T&`.

Objekat na koji funkcija vraća referencu mora postojati i onda kada se rezultat poziva upotrebi. Referenca na lokalnu promenljivu funkcije zato se ne sme vraćati:

```
int& neispravno()
{
    int x = 10;
    return x;    // greska: x prestaje da postoji pri izlasku iz funkcije
}
```

Po završetku funkcije objekat *x* više ne postoji, pa vraćena referenca ostaje bez objekta; takvu referencu nazivamo *visećom referencom*. U funkciji *element* toga nema, jer *element* kome se pristupa pripada nizu koji nije lokalni za tu funkciju i povratak iz funkcije ne završava njegov životni vek. Viseće reference i viseći pokazivači imaju isti uzrok, pa im se zajedno vraćamo u odeljku 2.5.

2.2.4 Kada pokazivač, a kada referenca

Referenca je, dakle, bezbednija: uvek označava neki objekat i ne može se slučajno preusmeriti. Otuda i pravilo koje se često čuje, da u savremenom C++ treba koristiti referencu kad god je to moguće. Pokazivači ipak nisu ostatak starijeg načina programiranja i ne mogu se izbeći u strukturama podataka, jer upravo svojstva koja referencu čine bezbednom istovremeno je čine i neupotrebljivom u tri situacije.

Prva je odsustvo objekta. Poslednji čvor liste nema sledbenika, prazno stablo nema koren, a pretraga koja nije uspeła nema šta da vrati. Sve to su ispravna stanja strukture, a ne greške, i izražavaju se vrednošću *nullptr*. Referenca nema odgovarajuće prazno stanje, pa njome ta stanja ne mogu da se opišu.

Druga je promena veze. Umetanje i brisanje u povezanim strukturama sastoje se upravo od preusmeravanja veza među čvorovima, a kod dinamičkog niza se pri realokaciji ceo blok zamenjuje novim. Član čvora koji upućuje na sledeći čvor zato mora da se menja tokom života strukture, dok referenca ostaje zauvek vezana za objekat sa kojim je nastala.

Treća je dinamički životni vek. Objekti koje struktura pravi tokom izvršavanja moraju da nadžive funkciju koja ih je napravila, pa se ne mogu čuvati kao lokalne promenljive. Takvi objekti se zauzimaju u dinamičkoj memoriji, a operacija koja ih zauzima daje njihovu adresu, koja se čuva u pokazivaču. Tome je posvećen odeljak 2.4.

Podela poslova je stoga sledeća. U interfejsu strukture, tamo gde je objekat poznat i postoji, koriste se reference: `const T&` za parametre koje funkcija samo čita i `T&` za pristup elementu koji sme da se menja. Unutar strukture, gde objekat može da nedostaje, gde veza mora da se menja i gde

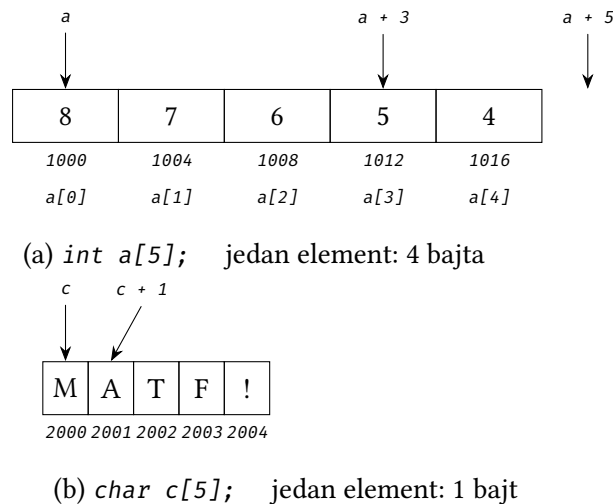
se objekti zauzimaju tokom izvršavanja, koriste se pokazivači. Obe uloge sreću se već u prvoj klasi koju pišemo, dinamičkom nizu iz glave 3: element se prosleđuje kao *const T&*, *operator[]* vraća *T&*, a blok u kome su elementi smešteni pamti se pokazivačem.

2.3 Statički niz i njegova ograničenja

Rad sa nizom fiksne dužine poznat je iz dosadašnjeg programiranja. Ono što ovde razmatramo jeste način na koji su njegovi elementi smešteni u memoriju, jer iz tog rasporeda slede i cena pristupa elementu i veza niza sa pokazivačima, na kojoj počivaju sve strukture u nastavku. Nazivom *statički niz* označavamo ugrađeni niz čiji je broj elemenata određen pri prevođenju; naziv se odnosi na fiksnu veličinu, a ne na ključnu reč *static*.

2.3.1 Raspored elemenata i adresa

Deklaracijom *int a[5];* obezbeđuje se prostor za pet celih brojeva. Elementi su smešteni u uzastopnim bajtovima memorije: odmah iza prostora jednog elementa počinje prostor sledećeg, bez razmaka između njih. Takav raspored nazivamo *kontinualnim* (eng. *contiguous*), i on je prikazan na slici 2.5.



Slika 2.5: Kontinualni raspored elemenata niza. (a) Kod niza celih brojeva susedni elementi su udaljeni četiri bajta, pa izraz $a + 3$ označava element $a[3]$, a $a + 5$ mesto neposredno iza poslednjeg elementa. (b) Kod niza karaktera isti pomeraj za jedan element znači pomeraj za jedan bajt. Adrese su ilustrativne.

Ako se početna adresa niza označi sa A , a veličina jednog elementa u bajtovima sa s , adresa elementa sa indeksom i jednaka je

$$\text{adresa}(a[i]) = A + i \cdot s, \quad 0 \leq i < n,$$

gde je n broj elemenata. Za niz koji počinje na adresi 1000, uz $\text{sizeof}(\text{int}) = 4$, element $a[3]$ počinje na adresi 1012.

Da bi se pristupilo elementu $a[i]$, nije potrebno obići elemente koji mu prethode: njegova adresa se izračuna iz početne adrese, indeksa i i veličine elementa, pa pristup po indeksu ima složenost $O(1)$. Ocena se odnosi na pristup elementu čiji je indeks poznat; pronalaženje elementa sa zadatom vrednošću posebna je operacija, koja u opštem slučaju zahteva obilazak niza.

Veličina elementa koja u formuli figuriše potiče iz tipa, pa se odatle dobija i ukupna veličina niza: `sizeof(a)` daje $n \cdot s$, a broj elemenata može se izračunati kao `sizeof(a) / sizeof(a[0])`. Uz same elemente ne čuva se nikakav dodatni podatak o dužini, pa statički niz svoju dužinu ne zna u smislu podatka koji bi se mogao pročitati tokom izvršavanja. To je prva razlika u odnosu na strukture koje gradimo počev od glave 3.

2.3.2 Ime niza i pokazivačka aritmetika

U većini izraza ime niza se pretvara u pokazivač na njegov prvi element, pa pokazivači p i q u nastavku dobijaju istu vrednost:

```
int a[5] = {};  
int* p = a;  
int* q = &a[0];
```

Niz, međutim, nije pokazivačka promenljiva. Deklaracijom niza obezbeđen je prostor za pet celih brojeva, dok pokazivač p čuva jednu adresu i može se preusmeriti. Ime niza se ne može preusmeriti, pa naredbe oblika $a = p$; ili $++a$; nisu dozvoljene.

Pokazivač se pomera po elementima, a ne po bajtovima. Ako p pokazuje na element tipa T , izraz $p + k$ označava mesto udaljeno $k * \text{sizeof}(T)$ bajtova, zbog čega isti zapis nad nizom celih brojeva i nad nizom karaktera pomera pokazivač za različit broj bajtova, kao na slici 2.5. Prevodilac veličinu elementa uzima iz tipa pokazivača, pa se ona nigde ne piše ručno. Otuda i veza indeksiranja sa pokazivačkom aritmetikom: za važeći indeks i izrazi $a[i]$ i $*(a + i)$ označavaju isti element, a $&a[i]$ i $a + i$ njegovu adresu. Indeksni zapis sme da stoji i uz pokazivač, pa $p[1]$ znači isto što i $*(p + 1)$.

Pored sabiranja sa celim brojem, pokazivači se pomeraju operatorima $++$, $--$, $+=$ i $-=$, a dva pokazivača na elemente istog niza mogu se oduzeti i uporediti. Razlika $q - p$ daje razliku indeksa, a ne broj bajtova između njih.

Za niz od n elemenata dozvoljeno je formirati pokazivače na elemente i pokazivač $a + n$, koji označava mesto neposredno iza poslednjeg elementa. Taj pokazivač služi kao oznaka kraja i sme da se poredi, ali ne i da se dereferencira. Niz se zato može obići i bez indeksa:

```
for (int* p = a; p != a + 5; ++p)  
    std::cout << *p << ' ';
```

Formiranje pokazivača izvan tog opsega, kao $a - 1$ ili $a + 6$, nije dozvoljeno ni kada se takav pokazivač ne dereferencira.

Pri kombinovanju pomeranja i dereferenciranja pazi se na prioritet operatora: izraz $*p + 1$ uvećava pročitano vrednost, $*(p + 1)$ pristupa sledećem elementu, $(*p)++$ uvećava element, a $*p++$ čita element na koji p pokazuje, a zatim pomera p na sledeći, što je verovatno ono što nismo želeli da se desi.

Navedene operacije objedinjuje sledeći program.

```
int a[] = {8, 7, 6, 5, 4, 3, 2, 1};  
  
std::cout << a[0] << ' ' << a[3] << ' ' << a[5] << '\n';  
  
int* p1 = a;  
int* p2 = &a[3];
```

```

int* p3 = a + 5;
std::cout << *p1 << ' ' << *p2 << ' ' << *p3 << '\n';

std::cout << p1[1] << ' ' << p2[2] << ' ' << p3[-1] << '\n';

++p1;
--p2;
p3 += 2;
std::cout << *p1 << ' ' << *p2 << ' ' << *p3 << ' ' << p3 - p1 << '\n';

int v = *p1++;
std::cout << v << ' ' << *p1 << '\n';

(*p1)++;
std::cout << *p1 << ' ' << a[2] << '\n';

```

Njegov izlaz je

```

8 5 3
8 5 3
7 3 4
7 6 1 6
7 6
7 7

```

Prva dva reda ispisa pokazuju da se istim elementima pristupa i preko indeksa i preko pokazivača. Treći red pokazuje da se indeksiranje računa u odnosu na trenutni položaj pokazivača, pa `p2[2]` i `p3[-1]` označavaju elemente `a[5]` i `a[4]`. U četvrtom redu poslednja vrednost je razlika pokazivača, dakle razlika indeksa elemenata na koje oni ukazuju, a ne broj bajtova. Peti red odgovara izrazu `*p1++`: pročitana je vrednost elementa `a[1]`, a pokazivač je pomeren na `a[2]`. U poslednjem redu izraz `(*p1)++` menja sam element, što se vidi i po tome što je promenu zabeležio i niz `a`.

Vredi naglasiti i granicu ovog računa. Pokazivačka aritmetika oslanja se na kontinualni raspored i ima smisla samo unutar jednog niza. Kod struktura čiji elementi nisu susedni u memoriji, kakve su povezane liste iz glave ??, izraz `++p` ne vodi do sledećeg elementa strukture, nego do susednog mesta u memoriji, koje strukturi najčešće i ne pripada. Do sledećeg elementa se tamo dolazi isključivo preko pokazivača koji je u strukturi sačuvan.

2.3.3 Nizovi kao argumenti funkcija

Parametar koji izgleda kao niz zapravo je pokazivač, pa sledeće dve deklaracije opisuju istu funkciju:

```

void obradi(int x[], int n);
void obradi(int* x, int n);

```

Pri pozivu `obradi(a, 5)`; funkciji se prenosi adresa prvog elementa, a ne kopija niza, pa se preko parametra `x` pristupa elementima izvornog niza. Broj elemenata mora se preneti zasebno, ovde parametrom `n`, jer se iz pokazivača ne može rekonstruisati dužina niza: u telu funkcije `sizeof(x)` daje veličinu pokazivača. Pošto se prenosi adresa, funkciji se sme proslediti i deo niza, na primer pozivom `obradi(a + 2, 3)`;

2.3.4 Granice niza

Ugrađeni operator indeksiranja ne proverava da li indeks pripada dozvoljenom opsegu. Za niz `int a[5]` pristup preko `a[5]` ili `a[100]` izlazi iz njegovih granica i dovodi do nedefinisanog ponašanja. Program pri tome ne mora da se prekine: pristup može pogoditi memoriju drugog objekta i neprimetno oštetiti njegove podatke, pa poruka *Segmentation fault* nije pouzdan način da se takva greška otkrije. Provera indeksa je posao strukture koja niz koristi, o čemu je reč u odeljku 3.2.

Broj elemenata statičkog niza, najzad, poznat je pri prevođenju i zadaje se konstantnim izrazom; ako se vrednost n učitava tokom izvršavanja, deklaracija `int podaci[n];` nije deo standardnog C++. Lokalni niz uz to prestaje da postoji pri izlasku iz bloka u kome je deklarisan. Time su elementi strukture vezani za prostor koji je određen pre pokretanja programa, pa nam je potreban način da se memorija zauzme tokom izvršavanja i da se, kada zatreba, zameni većom. Tome je posvećen naredni odeljak.

2.4 Dinamička alokacija

Broj podataka koje program obrađuje često nije poznat dok se program piše: on zavisi od korisničkog unosa, sadržaja datoteke ili ishoda prethodnog izračunavanja. Statički niz traži da se taj broj odredi pri prevođenju, pa se bira neka gornja granica, koja je ili premala za veće ulaze ili prevelika za manje unose. *Dinamička alokacija memorije* (eng. *dynamic memory allocation*) uklanja taj izbor: program prostor traži tokom izvršavanja, kada su mu potrebni podaci već poznati, i vraća ga kada mu više nije potreban. U jeziku C++ za zauzimanje prostora koristi se izraz `new`, a za njegovo oslobađanje izraz `delete`; za nizove postoje oblici `new[]` i `delete[]`.

2.4.1 Automatska i dinamička memorija

Lokalna promenljiva, kakav je i niz iz prethodnog odeljka, ima *automatsko trajanje*: nastaje pri izvršavanju svoje deklaracije, a prestaje da postoji pri izlasku iz bloka, bez ijedne naredbe koju bi programer za to napisao. Prostor za takve objekte obezbeđuje se na *programskom steku*, koji prati pozive funkcija, pa svako aktivno izvršavanje funkcije ima sopstvene lokalne objekte.

Objekat zauzet izrazom `new` ima *dinamičko trajanje*: nastaje izvršavanjem tog izraza i postoji dok ga izraz `delete` ne uništi, nezavisno od bloka u kome je nastao. Prostor za njega obezbeđuje se iz oblasti koju nazivamo *dinamička memorija* (eng. *heap*).¹ Stek i dinamička memorija model su koji objašnjavaju ponašanje programa; standard jezika ne propisuje njihov fizički raspored.

Razlika je, dakle, u tome ko određuje kraj života objekta. Automatski objekat nestaje sa svojim blokom, dok se dinamički objekat može koristiti i pošto se funkcija koja ga je zauzela završi, pod uslovom da je sačuvan pokazivač na njega. Time nastaje i obaveza koje kod automatskih objekata nema: svaki zauzeti prostor mora se osloboditi.

2.4.2 Alokacija pojedinačnog objekta

Izraz `new` navodi tip objekta i, po potrebi, njegov inicijalizator:

```
int* p = new int(10);
```

¹Engleski naziv *heap* koristi se i za strukturu podataka o kojoj je reč u glavi ??, a koju nazivamo hip. Dva pojma nisu povezana: ovde je reč o oblasti memorije, tamo o organizaciji podataka.

Zauzima se prostor za jedan objekat tipa *int*, objekat se inicijalizuje vrednošću 10, a rezultat izraza je adresa tog objekta, tipa *int**. Veličinu određuje tip, pa se broj bajtova nigde ne navodi. Dinamički objekat nema sopstveno ime: njemu se pristupa isključivo preko pokazivača, izrazom **p*.

Način inicijalizacije razlikuje se kao i kod ostalih objekata:

```
int* a = new int;      // vrednost nije inicijalizovana
int* b = new int();    // vrednost je 0
```

Sama alokacija, dakle, ne podrazumeva postavljanje na nulu. Kada objekat više nije potreban, oslobađa se:

```
delete p;
p = nullptr;
```

Naredba *delete p*; uništava objekat i oslobađa njegov prostor, ali ne dira samu pokazivačku promenljivu *p*, koja i dalje sadrži staru adresu. Dodela *nullptr* zato je zasebna naredba, kojom se beleži da *p* više nema objekat. Oslobođeni prostor postaje raspoloživ za naredne alokacije, što ne znači da se odmah vraća operativnom sistemu.

2.4.3 Dinamička alokacija nizova

Niz se zauzima oblikom *new T[n]*, pri čemu broj elemenata *n* ne mora da bude poznat pri prevođenju, nego tek u trenutku kada se alokacija izvrši:

```
int n = 4;
int* p = new int[n]{};
p[2] = 7;

delete[] p;
```

Izraz daje pokazivač na prvi element, a elementi leže u kontinualnom prostoru, pa za njih važe ista pravila indeksiranja i pokazivačke aritmetike kao za statički niz, uključujući i pristup po indeksu složenosti $O(1)$. Oblik *new int[n]* ne postavlja elemente na nulu; zagrade *{}* u primeru upravo to čine.

Slika 2.6 razdvaja dve vrste memorije koje se u ovom primeru sreću. Promenljive *n* i *p* su lokalne i imaju automatsko trajanje, dok niz na koji *p* pokazuje ima dinamičko trajanje. Tip pokazivača ne govori ništa o tome kako je memorija cilnog objekta obezbeđena: pokazivač tipa *int** jednako pokazuje na element lokalnog i na element dinamički zauzetog niza.

Pokazivač ne sadrži podatak o broju elemenata, pa *sizeof(p)* daje veličinu pokazivača, a ne veličinu niza. Broj elemenata čuva se zasebno, kao ovde u promenljivoj *n*. Pri oslobađanju se, nasuprot tome, piše samo *delete[] p*;, bez broja elemenata, jer taj podatak upravljač memorijom vodi sam.

2.4.4 Uparivanje alokacije i oslobađanja

Oblik oslobađanja mora da odgovara obliku alokacije:

Zauzimanje	Oslobađanje
<i>T* p = new T;</i>	<i>delete p;</i>

Zauzimanje	Oslobađanje
$T^* p = \text{new } T[n];$	$\text{delete}[] p;$

Pravilo važi i za nizove osnovnih tipova i za niz od jednog elementa; upotreba jednog oblika umesto drugog dovodi do nedefinisanog ponašanja. Uz to, izrazu `delete[]` prosleđuje se pokazivač dobijen alokacijom, dakle pokazivač na prvi element, a ne na neki unutrašnji, poput $p + 1$. Automatski objekat se ne sme oslobađati, niti se isti dinamički objekat sme osloboditi dvaput, dok je primena izraza `delete` na nulti pokazivač dozvoljena i ne radi ništa.

Za korisnički definisane tipove alokacija i oslobađanje obuhvataju i konstruisanje i uništavanje objekata, što pokazuje sledeći program:

```
struct Tacka {
    Tacka() { std::cout << '+'; }
    ~Tacka() { std::cout << '-'; }
};

Tacka* niz = new Tacka[5];
delete[] niz;
```

Njegov izlaz je `+++++-----`: izraz `new Tacka[5]` poziva podrazumevani konstruktor za svaki element, a `delete[] niz` destruktore svakog elementa, i to obrnutim redosledom od redosleda konstruisanja. Odatle i zahtev da tip elementa ima podrazumevani konstruktor, na koji se oslanja dinamički niz u glavi 3.

2.4.5 Primer: broj elemenata poznat tek tokom izvršavanja

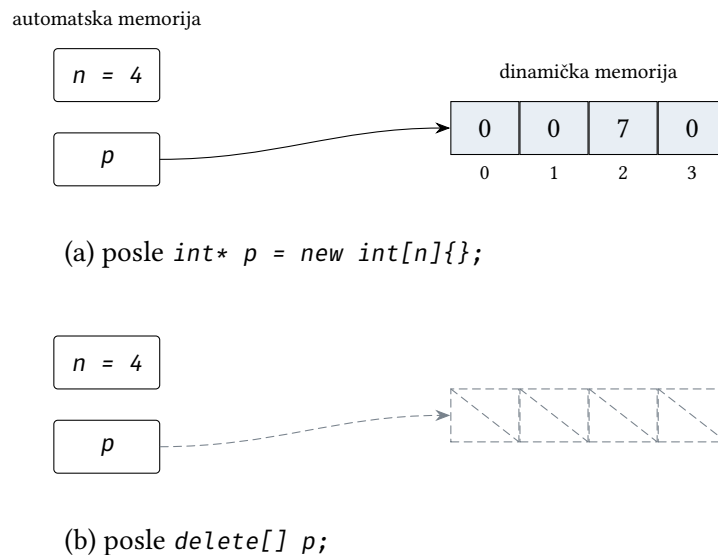
Sledeći program učitava broj elemenata, obezbeđuje prostor za njih, učitava elemente i ispisuje ih u obrnutom poretku. Pri prevođenju nije potrebno znati koliko će elemenata biti uneto.

```
int main()
{
    int n;
    if (!(std::cin >> n) || n <= 0) {
        std::cerr << "Broj elemenata mora biti pozitivan.\n";
        return 1;
    }

    int* a = new int[n];

    for (int i = 0; i < n; ++i) {
        if (!(std::cin >> a[i])) {
            std::cerr << "Neispravan unos elementa.\n";
            delete[] a;
            return 1;
        }
    }

    for (int i = n - 1; i >= 0; --i)
```



Slika 2.6: Automatska i dinamička memorija. (a) Lokalne promenljive n i p postoje dok traje blok u kome su deklarisanе, a niz od četiri elementa zauzet je u dinamičkoj memoriji. (b) Posle `delete[]` p elementi više ne postoje, dok lokalne promenljive i dalje postoje; pokazivač p pri tome zadržava staru adresu.

```
std::cout << a[i] << ' ';
std::cout << '\n';

delete[] a;
return 0;
}
```

Za ulaz 5 i elemente 8 7 6 5 4 program ispisuje 4 5 6 7 8. Vredi uočiti i oslobađanje pri pre-vremenom izlasku: svaki put kada se funkcija napusti, niz mora biti oslobođen, pa se ista naredba ponavlja na dva mesta. Kod složenijih funkcija sa više izlaza to postaje glavni izvor grešaka, a rešenje koje ga uklanja opisano je u odeljku 2.6.

Neuspeh alokacije program ne proverava poređenjem sa `nullptr`: uobičajeni oblik izraza `new` pri nedostatku memorije prijavljuje izuzetak tipa `std::bad_alloc` iz zaglavlja `<new>`.

2.4.6 Veličina zauzetog niza se ne menja

Mogućnost da se dužina odredi tokom izvršavanja ne znači da već zauzet niz može da raste. Broj elemenata određen je vrednošću izraza u uglastim zagradama u trenutku alokacije, pa kasnija promena promenljive n ne menja veličinu niza:

```
int n = 4;
int* p = new int[n]{};

n = 10; // menja se samo n; niz i dalje ima cetiri elementa
// p[4] = 7; // neispravno: pristup van granica niza
```

Ako zatreba više mesta, zauzima se novi niz, postojeći elementi prenose se u njega, a stari se oslobađa. Taj postupak, zajedno sa pitanjem koliko novog prostora tražiti, čini osnovu dinamičkog niza iz glave 3.

Dinamička alokacija, dakle, daje kontrolu nad trenutkom nastanka objekta, nad količinom zauzetog prostora i nad trenutkom oslobađanja. Uz tu kontrolu ide i obaveza da se oslobađanje zaista i izvrši, tačno jednom i u pravom trenutku. Šta se dešava kada se ta obaveza prekrši, tema je narednog odeljka.

2.5 Životni vek objekata

Za ispravan rad sa pokazivačem nije dovoljno znati adresu objekta i njegov tip; potrebno je znati i da li taj objekat još postoji. Pokazivač može ostati upotrebljiv i pošto objekat na koji je ukazivao prestane da postoji, a dinamički objekat može nastaviti da postoji i pošto nestane pokazivač preko koga je nastao. Greške koje odatle slede spadaju među najčešće u radu sa strukturama podataka, pa su predmet ovog odeljka.

Životni vek (eng. *lifetime*) objekta je period izvršavanja u kome objekat postoji. Treba ga razlikovati od *dosega* (eng. *scope*) imena, koji je deo izvornog koda u kome se ime može upotrebiti. Kod lokalne promenljive ta dva pojma se poklapaju, pa deluju kao jedno, ali se u opštem slučaju razlikuju: dinamički objekat nema sopstveno ime, a dostupan je preko svakog pokazivača koji čuva njegovu adresu.

2.5.1 Automatski i dinamički životni vek

Lokalna promenljiva bez oznake *static* ima automatsko trajanje: nastaje kada izvršavanje stigne do njene deklaracije, a prestaje da postoji pri napuštanju bloka, uključujući i izlazak naredbom *return*.

```
void primer()
{
    int spoljasnji = 10;
    {
        int unutrasnji = 20;
        spoljasnji += unutrasnji;
    } // završava se životni vek objekta unutrasnji
} // završava se životni vek objekta spoljasnji
```

Svaki poziv funkcije ima sopstvene automatske objekte, pa lokalna promenljiva jednog aktivnog poziva i istoimena promenljiva drugog poziva iste funkcije nisu isti objekat. To važi i za rekurzivne pozive.

Dinamički objekat, nastao izrazom *new*, ne prestaje da postoji pri napuštanju bloka, nego tek kada se nad njim izvrši *delete*. Zato on može da nadživi funkciju koja ga je napravila, što je osobina na kojoj počivaju sve strukture u nastavku: čvor koji funkcija umetanja napravi mora da ostane i pošto se ta funkcija završi. Ta mogućnost postoji samo za dinamičke objekte. Funkcija ne može da održi u životu svoju lokalnu promenljivu time što će vratiti pokazivač ili referencu na nju.

2.5.2 Konstruktor i destruktor

Kod klasnih tipova, među koje spadaju i tipovi definisani pomoću *struct*, *konstruktor* određuje šta se radi pri nastanku objekta, a *destruktor* šta se radi pri njegovom uništavanju. Konstruktor nosi ime klase, ime destruktora dobija se dodavanjem znaka ~ ispred tog imena, i nijedan nema povratni tip. Redosled njihovog pozivanja prati sledeći program:

```

struct Trag {
    int oznaka;

    Trag(int k) : oznaka(k) { std::cout << "Konstruktor " << oznaka << '\n'; }
    ~Trag()                { std::cout << "Destruktor " << oznaka << '\n'; }
};

int main()
{
    Trag a(1);
    {
        Trag b(2);
        Trag* p = new Trag(3);
        delete p;
    }
}

```

Zapis : *oznaka(k)* inicijalizuje član *oznaka* pre izvršavanja tela konstruktora. Program ispisuje:

```

Konstruktor 1
Konstruktor 2
Konstruktor 3
Destruktor 3
Destruktor 2
Destruktor 1

```

Automatski objekti *a* i *b* uništavaju se pri napuštanju svojih blokova, i to obrnutim redosledom od redosleda nastanka. Za dinamički objekat, destruktore se poziva naredbom *delete*, a ne izlaskom iz bloka. Da je *delete p*; izostavljen, objekat (koji je nastao sa *new*) ostaje u memoriji bez ijednog pokazivača preko koga bi mu se moglo pristupiti ili preko koga bi se mogao osloboditi.

2.5.3 Vlasništvo

Dinamički objekat ne nestaje sam. Neko mora da napiše *delete*, i to tačno jednom, tek kada objekat više nikome nije potreban. Deo programa koji je za to zadužen nazivamo *vlasnikom* objekta, a njegovu odgovornost *vlasništvom* (eng. *ownership*).

Zato pokazivač može da bude u jednoj od dve uloge:

- *vlasnički pokazivač* – preko njega se objekat oslobađa;
- *pozajmljeni pokazivač* – preko njega se objekat samo koristi.

Funkcija koja dobije pokazivač na niz da bi ispisala njegove elemente ima pozajmljen pokazivač: ona taj niz čita, ali ga ne oslobađa. Kada bi ga oslobodila, deo programa koji joj ga je prosledio ostao bi sa višećim pokazivačem.

Po tipu se te dve uloge ne razlikuju, jer su oba pokazivača tipa *int**. Uloga se, dakle, ne vidi iz tipa, pa mora biti dogovorena i zapisana u opisu funkcije ili klase.

U strukturama koje gradimo dogovor je uvek isti: struktura je vlasnik sve memorije koju zauzme, a oslobađa je njen destruktore. Svi ostali delovi programa dobijaju pozajmljene pokazivače.

2.5.4 Curenje memorije

Curenje memorije (eng. *memory leak*) nastaje kada program ne oslobodi dinamičku memoriju koja mu više nije potrebna.

```
int* p = new int[100];
// koriscenje prvog niza

p = new int[200]; // izgubljena je adresa prvog niza
// koriscenje drugog niza

delete[] p;      // oslobadja se samo drugi niz
```

Dodela pokazivaču *p* menja samo njegovu vrednost, a prvi niz ostaje zauzet i nedostupan. Isto nastaje i kada se funkcija završi pre planiranog *delete[]*, na primer zbog ranog *return* ili izuzetka.

Curenje ne mora odmah da promeni rezultat programa. Ako se ponavlja tokom dugotrajnog rada, količina memorije koja više nije dostupna raste i može dovesti do iscrpljivanja raspoloživog prostora, što može biti uočeno kasno.

2.5.5 Viseći pokazivači i dvostruko oslobađanje

Viseći pokazivač (eng. *dangling pointer*) jeste pokazivač koji je ukazivao na objekat čiji je životni vek završen. Može nastati i bez dinamičke alokacije:

```
int* p = nullptr;
{
    int x = 7;
    p = &x;
}
// p je viseći pokazivac: x vise ne postoji
```

Čuvanje adrese, dakle, ne održava objekat u životu. Isto važi i za reference, o čemu je bilo reči u odeljku 2.2.

Kod dinamičkih objekata viseći pokazivači mogu nastati i posle oslobađanja, često kada na isti objekat ukazuje više pokazivača:

```
int* p = new int(7);
int* q = p;

delete p;
p = nullptr;

// q je dalje viseći pokazivac
// std::cout << *q; // greska: objekat vise ne postoji
// delete q;       // greska: dvostruko oslobađanje
```

Izraz *delete p* ne postavlja *p* na *nullptr* niti menja ostale pokazivače na isti objekat. Dodela *p = nullptr*; beleži da *p* ne pokazuje ni na jedan objekat, ali za *q* to ne čini ništa. Ponovno oslobađanje istog objekta, *dvostruko oslobađanje*, dovodi do nedefinisanog ponašanja; primena izraza *delete* na nulti pokazivač, nasuprot tome, dozvoljena je i ne radi ništa.

Pristup preko visećeg pokazivača ponekad prekine izvršavanje programa uz poruku *Segmentation fault*. Do prekida dolazi kada program pokuša da pristupi adresi koja mu nije dodeljena; operativni sistem to otkriva i zaustavlja program, a takav prekid nazivamo *segmentacijska greška* (eng. *segmentation fault*). Ova poruka zapravo potiče od operativnog sistema.

Na tu poruku se, međutim, ne treba osloniti. Oslobođeni blok i dalje pripada programu, pa pristup njegovim bajtovima operativni sistem najčešće i ne primećuje. Program tada nastavlja rad sa vrednostima koje su slučajno ostale na toj adresi, ili korišćenjem visećeg pokazivača ošteti podatke nekog drugog objekta, a greška se ispolji mnogo kasnije i na sasvim drugom mestu.

2.5.6 Kopiranje pokazivača nije kopiranje objekta

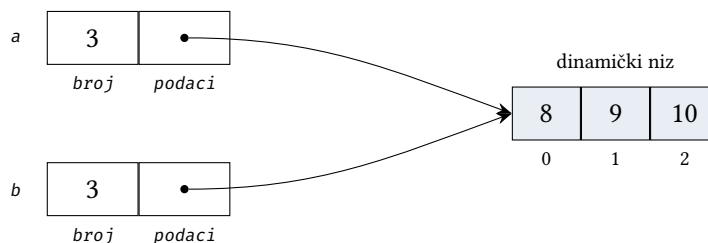
Pitanje vlasništva naročito dolazi do izražaja pri kopiranju struktura koje sadrže pokazivač. Posmatrajmo strukturu za čuvanje ocena:

```
struct Ocene {
    int broj;
    int* podaci;
};

Ocene a{3, new int[3]{8, 9, 10}};
Ocene b = a;

b.podaci[0] = 6; // i a.podaci[0] sada ima vrednost 6
```

Podrazumevano kopiranje kopira svaki član, pa objekti *a* i *b* imaju zasebne članove *broj* i zasebne pokazivače *podaci*. Oba pokazivača, međutim, sadrže istu adresu, pa su nastala dva objekta tipa *Ocene* nad jednim jedinim nizom ocena, kao na slici 2.7.



Slika 2.7: Stanje posle inicijalizacije *Ocene b = a*. Objekti *a* i *b* imaju zasebne članove, ali oba člana *podaci* ukazuju na isti dinamički niz.

Takvo kopiranje naziva se *plitko kopiranje* (eng. *shallow copy*): kopira se adresa, a ne objekat na toj adresi. Otuda i posledice koje se u primeru vide. Izmena preko *b.podaci* vidljiva je i preko *a.podaci*. Oslobođanje niza izrazom *delete[] a.podaci* ostavlja *b.podaci* kao viseći pokazivač, a naknadno *delete[] b.podaci* bilo bi dvostruko oslobođanje. Uništavanje samih objekata *a* i *b* pri tome niz uopšte ne oslobađa, jer ova struktura nema destruktora.

Ako svaki objekat treba da ima sopstvene ocene, pri kopiranju se mora zauzeti novi niz i u njega prepisati elementi. To je *duboko kopiranje* (eng. *deep copy*), čija se implementacija, zajedno sa pravilima kopiranja objekata koji upravljaju memorijom, razmatra u odeljku 3.5.

Otkrivanje grešaka. Nedozvoljen pristup memoriji i dvostruko oslobođanje otkrivaju alati kao što su AddressSanitizer i Valgrind, a curenje memorije LeakSanitizer i Valgrind.

AddressSanitizer se uključuje pri prevođenju, opcijom `-fsanitize=address`, posle čega program sam prijavljuje greške dok radi. Ti alati prate ono što se u jednom izvršavanju stvarno dogodi. Ako neka grana programa tada nije izvršena, greške u njoj ostaju neprijavljene, pa program koji je „prošao bez prijave” nije time dokayano proveren.

Greške opisane u ovom odeljku imaju zajednički uzrok: oslobađanje je zasebna naredba koju programer mora sam da napiše, na svakom mestu na kome funkcija može da se završi, i to tačno jednom. Naredni odeljak uvodi način da se ta obaveza veže za životni vek objekta koji je za memoriju odgovoran.

2.6 Osnovna ideja RAI

U prethodnom odeljku videli smo da izlazak iz bloka ne oslobađa dinamičku memoriju. Programer mora sam da napiše `delete`, i to na svakom mestu na kome funkcija može da se završi. Jedna stvar se, međutim, dešava sama: kada se blok napusti, pozivaju se destruktori lokalnih objekata. Na tome počiva rešenje. Memoriju poveravamo objektu koji ima destruktork, a destruktork je zatim oslobađa umesto nas.

Taj postupak naziva se *RAI* (eng. *Resource Acquisition Is Initialization*). Pravilo je kratko: konstruktor zauzima resurs, destruktork ga oslobađa. Resurs je najčešće memorija, ali može biti i otvorena datoteka ili nešto drugo što se posle upotrebe mora zatvoriti ili vratiti.

Vratimo se strukturi *Ocene* iz odeljka 2.5. Niz ocena zauzima onaj ko pravi objekat, a oslobađa ga onaj ko se poslednji njime koristi, pa se na to lako zaboravi. Da struktura ima destruktork koji oslobađa niz, tog dogovora ne bi ni bilo: niz bi nestajao zajedno sa objektom, na svakom izlasku iz funkcije i pri izuzetku. Odatle pravilo koje u knjizi važi za svaku klasu koja sama zauzima memoriju: **ona mora imati destruktork koji tu memoriju oslobađa**. Bez destruktorka svaki njen objekat ostavlja za sobom zauzetu memoriju. Prva takva klasa je dinamički niz iz glave 3, koji memoriju zauzima u konstruktoru, a oslobađa u destruktorku.

2.6.1 Pravilo trojke i pravilo petorke

Destruktork nije jedina operacija o kojoj treba voditi računa. Objekat se može i kopirati, a podrazumevano kopiranje kopira pokazivač, ne i niz na koji on ukazuje. Ako bi struktura *Ocene* dobila destruktork, a kopiranje ostalo podrazumevano, dva objekta bi delila isti niz i oba bi ga na kraju oslobodila. Otuda smernica poznata kao *pravilo trojke* (eng. *rule of three*): klasa kojoj je potreban sopstveni destruktork po pravilu mora da definiše i konstruktor kopije i operator dodele. Te tri operacije zajedno određuju šta se dešava pri uništavanju i pri kopiranju objekta koji poseduje resurs. Savremeni C++ dodaje im još dve, konstruktor premeštanja i operator dodele premeštanjem, pa se proširena smernica naziva *pravilo petorke* (eng. *rule of five*).

Od tih operacija u knjizi pišemo destruktork, dok kopiranje zabranjujemo deklaracijama oblika

```
Klasa(const Klasa&) = delete;
Klasa& operator=(const Klasa&) = delete;
```

Zabrana je namerna: struktura koja se ne kopira ne može ni da izazove dvostruko oslobađanje. Premeštanje ostavljamo po strani.

2.6.2 Pametni pokazivač `std::unique_ptr`

Za pojedinačni dinamički objekat standardna biblioteka već nudi tip koji radi po pravilu RAI: `std::unique_ptr<T>` iz zaglavlja `<memory>`. Takav tip naziva se *pametni pokazivač* (eng. *smart pointer*), jer se koristi kao pokazivač, a ponaša se kao vlasnik.

```
void obradi(bool prekini)
{
    auto p = std::make_unique<int>(42);

    std::cout << *p << '\n';

    if (prekini)
        return;    // p se unistava i oslobadja ceo broj

    *p = 17;

    // i ovde p oslobadja ceo broj
}
```

Poziv `std::make_unique<int>(42)` zauzima ceo broj sa vrednošću 42 i predaje ga pokazivaču `p` na čuvanje. Objektu se pristupa kao i preko običnog pokazivača, izrazima `*p` i `p->clan`, a `if (p)` proverava da li pokazivač uopšte ima objekat.

U poređenju sa običnim pokazivačem, dobija se sledeće:

- `delete` se ne piše, pa se ne može ni zaboraviti, ni izostati na nekom od izlaza iz funkcije;
- vlasništvo se vidi iz tipa: `std::unique_ptr<T>` je vlasnik, a `T*` je pozajmljen pokazivač;
- kopiranje pametnog pokazivača prevodilac prijavljuje kao grešku, pa dva vlasnika nad istim objektom ne mogu ni da nastanu.

Cena je to što se vlasništvo ne prenosi dodelom, nego posebnom operacijom premeštanja, koju ne obrađujemo. Zbog toga strukture podataka u nastavku gradimo običnim pokazivačima, kod kojih se svaka veza menja jednostavnom dodelom.

U aplikativnom kodu. Strukture podataka u ovoj knjizi pišemo pomoću `new` i `delete`. Ipak, u svaremenom programiranju je preporuka da program koji takve strukture samo koristi, memoriju ne zauzima ručno. Umesto toga uzima gotov tip koji već radi po pravilu RAI: kontejner standardne biblioteke, kao što su `std::vector`, `std::map` ili `std::unordered_map` ili `std::unique_ptr<T>` za pojedinačni dinamički objekat. Klasa čiji su članovi takvi tipovi najčešće ne mora da ima sopstveni destruktork, jer se destruktorki članova pozivaju sami.

RAII, dakle, spaja dva zahteva. Svaki resurs ima tačno jednog vlasnika, a oslobađanje je deo uništavanja tog vlasnika, pa se dešava samo od sebe. U narednoj glavi po tom pravilu gradimo prvu strukturu podataka, dinamički niz, koji memoriju zauzima u konstruktoru i oslobađa je u destruktorku.

3

Dinamički niz

Niz je jedna od najjednostavnijih struktura za čuvanje uredene kolekcije elemenata i istovremeno jedna od najčešće korišćenih. Osobina zbog koje je toliko upotrebljiv jeste *direktan pristup* (eng. *random access*): ako je poznat indeks elementa, do njega se dolazi neposredno, bez prolaska kroz elemente koji mu prethode.

Nizovi kakvi su do sada korišćeni imaju dužinu koja se određuje prilikom prevođenja programa i ne menja se dok niz postoji. Deklaracija

```
int a[100];
```

obezbeđuje prostor za tačno 100 elemenata tipa *int*.

U velikom broju programa, međutim, broj elemenata koje treba sačuvati nije poznat u trenutku pisanja programa. Pretpostavimo da program učitava podatke sve dok se na ulazu ne pojavi oznaka kraja. Podataka može biti deset, sto ili nekoliko hiljada, i unapred se ne može pouzdano odrediti koliko.

Jedno rešenje jeste da se izabere dužina za koju verujemo da neće biti premala:

```
int a[10000];
```

Takvo rešenje uvodi dva problema odjednom. Ako program upotrebi samo nekoliko desetina elemenata, najveći deo rezervisanog prostora ostaje neiskorišćen; ako se pojavi više od deset hiljada elemenata, niz je nedovoljan i program ne može da nastavi. Pomeranje granice naviše ne otklanja nijedan od ta dva problema, nego samo bira koji će se češće javljati: svaka unapred izabrana konačna granica može se pokazati kao premala, a svaka velika granica troši memoriju koja najčešće neće biti upotrebljena.

Iz istog primera vidi se i da treba razlikovati dve veličine koje se lako mešaju: **broj elemenata koji se trenutno nalazi u kolekciji i količinu prostora rezervisanu za njih**. Kod niza fiksne dužine o toj razlici vodi računa sam programer, i to izvan niza:

```
int a[100];  
int n = 17;
```

Promenljiva *n* nije osobina niza *a*, nego dodatna informacija o načinu na koji ga program koristi. O njenoj ispravnosti brine kod koji niz koristi: pri svakom dodavanju i uklanjanju elementa *n* mora da se promeni, a nijedna operacija ne sme da prekorači fizičku granicu niza. Kompilator u tome ne pomaže, jer, kao što je pokazano u Glavi 2, proveru indeksa se ne vrši.

Prirodno je zato potražiti strukturu kod koje briga o raspoloživom prostoru nije odgovornost koda koji je koristi, nego same strukture: kada rezervisani prostor postane nedovoljan, struktura sama obezbeđuje veći. Takvu strukturu nazivamo **dinamički niz** (eng. *dynamic array*).

Posmatran sa strane programa koji ga koristi, dinamički niz je uređena kolekcija promenljive dužine, pri čemu broj elemenata ne mora unapred da bude poznat. Od njega očekujemo najmanje sledeće operacije:

- određivanje trenutnog broja elemenata;
- pristup elementu na osnovu indeksa;
- dodavanje novog elementa;
- uklanjanje postojećeg elementa.

Razlika u odnosu na niz fiksne dužine nije, dakle, u načinu na koji se elementima pristupa. Dinamički niz zadržava upravo onu osobinu zbog koje je niz koristan; menja se jedino to što količina prostora nije unapred fiksirana, već se prilagođava tokom izvršavanja programa.

Time se otvara pitanje koje kod niza fiksne dužine nije postojalo. Ako elementi i dalje treba da budu smešteni u uzastopnim memorijskim lokacijama - a treba, jer bez toga nema direktnog pristupa - jednom rezervisan blok ne može se proizvoljno proširiti, budući da memorija neposredno iza njega može već biti zauzeta. Kada u postojećem bloku nestane mesta, mora se pronaći drugi, veći blok, a postojeći elementi preneti u njega.

Zato implementacija dinamičkog niza mora u svakom trenutku da vodi računa o dvema veličinama: o broju elemenata koje niz sadrži i o broju elemenata za koje je prostor obezbeđen. Odnos tih dveju veličina određuje i način na koji dinamički niz radi i cenu njegovih operacija, pa time počinje naredni odeljak.

3.1 Kapacitet i veličina

Da bi dinamički niz mogao da menja dužinu tokom izvršavanja programa, nije dovoljno čuvati samo njegove elemente. Potrebno je znati i koliko se elemenata u nizu trenutno nalazi i za koliko je elemenata prostor obezbeđen. Prirodna reprezentacija dinamičkog niza sastoji se stoga od tri podatka:

- pokazivača na memorijski blok u kome se elementi čuvaju;
- *veličine*, odnosno broja elemenata koji se u nizu trenutno nalaze;
- *kapaciteta*, odnosno najvećeg broja elemenata koji u rezervisani blok mogu da stanu bez nove alokacije.

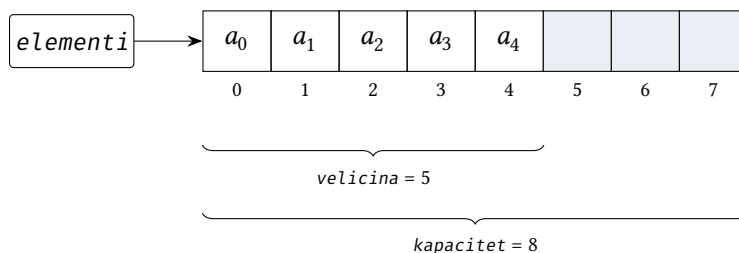
Klasa dinamičkog niza čiji su elementi tipa *int* sadržala bi tako sledeće članove:

```
class DinamickiNiz {
private:
    int* elementi;
    std::size_t velicina;
    std::size_t kapacitet;
};
```

Tip *std::size_t* je celobrojni neoznačen tip, dovoljno širok da izrazi veličinu bilo kog objekta u memoriji; njega vraća i operator *sizeof*, a deklarisan je u zaglavlju *<cstdlib>*. U ovoj knjizi ga koristimo svuda gde promenljiva označava broj elemenata ili poziciju u nizu, pošto se tako radi i u standardnoj biblioteci.

Član *elementi* ukazuje na početak kontinualnog bloka u kome su smešteni elementi niza. Prvih *velicina* pozicija tog bloka sadrži elemente koji nizu pripadaju, dok preostalih *kapacitet* -

velicina pozicija čini rezervisan, ali još neiskorišćen prostor. Veličina i kapacitet zato imaju različita značenja: ako je, na primer, veličina jednaka 5, a kapacitet 8, niz sadrži pet elemenata, a memorija je rezervisana za osam. Još tri elementa mogu se dodati u postojećem bloku; tek bi dodavanje devetog zahtevalo veći prostor.



Slika 3.1: Dinamički niz veličine 5 i kapaciteta 8. Osenčena polja pripadaju rezervisanom prostoru, ali ne i sadržaju niza.

Indeksi važećih elemenata su, kao i kod niza fiksne dužine, $0, 1, \dots, \text{velicina} - 1$. Kapacitet, međutim, nije deo logičkog sadržaja strukture: korisnika ne zanima da li niz od pet elemenata ima kapacitet 5, 8 ili 16, jer se ni u jednom od tih slučajeva sadržaj niza ne razlikuje. Kapacitet pripada unutrašnjoj reprezentaciji i postoji zato da bi izvestan broj budućih dodavanja mogao da se obavi bez ponovne alokacije.

Upravo to razdvajanje čini dinamički niz upotrebljivim. Kada bi se pri svakom dodavanju alocirao novi blok tačne dužine i u njega prepisali svi postojeći elementi, dodavanje jednog elementa koštalo bi onoliko koliko niz ima elemenata. Umesto toga, dinamički niz rezerviše više prostora nego što mu je u tom trenutku potrebno i taj prostor postepeno popunjava; koliko se time zaista dobija, pokazujemo u odeljku 3.3.

Vrednosti članova *elementi*, *velicina* i *kapacitet* ne mogu biti proizvoljne. One zajedno opisuju jedno stanje strukture, pa svaka operacija mora da očuva njihov međusobni odnos: veličina ne sme da premaši kapacitet, a blok na koji se ukazuje mora biti dovoljan za onoliko elemenata koliko kapacitet obećava. Uslov koji ovako mora da važi pre i posle svake ispravno izvršene operacije naziva se *invarijanta* strukture podataka (eng. *invariant*).

Invarijanta dinamičkog niza. U svakom ispravnom stanju važi $0 \leq \text{velicina} \leq \text{kapacitet}$, pri čemu *elementi* ukazuje na memorijski blok dovoljan za *kapacitet* elemenata, a pozicije $0, 1, \dots, \text{velicina} - 1$ tog bloka sadrže elemente niza.

Invarijanta nije samo opis; ona je merilo ispravnosti svake operacije koju u nastavku pišemo. Kod dinamičkog niza je jednostavna i lako se proverava, ali je isti pojam ono što kod složenijih struktura, poput stabala pretrage u glavi 9, uopšte omogućava da se govori o ispravnosti.

3.2 Klasa dinamičkog niza

Prethodni odeljak opisao je podatke koje dinamički niz mora da čuva: memorijski blok, trenutnu veličinu i kapacitet. Sada te podatke objedinjujemo u klasu i definišemo osnovne operacije nad njom.

Dinamički niz ne treba da bude vezan za jedan tip elemenata. Isti mehanizam upravljanja memorijom jednako radi za cele brojeve, realne brojeve, karaktere ili objekte proizvoljne klase, a jedino što se menja jeste tip onoga što se u bloku čuva. Zato klasu definišemo kao *šablon klase* (eng. *class template*).

```

template <typename T>
class DinamickiNiz {
private:
    T* elementi;
    std::size_t velicina_;
    std::size_t kapacitet_;

public:
    DinamickiNiz(std::size_t pocetni_kapacitet = 4);
    ~DinamickiNiz();

    std::size_t velicina() const;
    std::size_t kapacitet() const;

    T& operator[](std::size_t indeks);
    const T& operator[](std::size_t indeks) const;
};

```

Bez tog mehanizma, niz celih brojeva i niz realnih brojeva zahtevali bi dve klase čiji bi se kod razlikovao samo u jednoj reči. Ista klasa morala bi da se prepíše za svaki novi tip elemenata, a svaka ispravka da se ponovi u svakom prepisu. Šablon uklanja to ponavljanje tako što tip postaje *parametar* definicije.

Zapis `template <typename T>` označava da se definiše šablon i da je *T* ime kojim se u nastavku definicije označava *tip* koji još nije određen. Unutar klase *T* se koristi kao svako drugo ime tipa: njime se deklariraju članovi, tipovi argumenata i povratne vrednosti. Neformalno se `template <typename T>` može čitati kao „za svaki tip *T*“. Umesto reči *typename* sme da stoji i `class`, pa se u literaturi sreću oba zapisa; značenje je isto, a *T* ni u jednom slučaju ne mora biti klasa.

Tip se određuje tek pri upotrebi. Deklaracije

```

DinamickiNiz<int> a(10);
DinamickiNiz<double> b(20);

```

zato uvode dva različita tipa: prvi čuva elemente tipa *int*, drugi elemente tipa *double*. Tip *int*, odnosno *double* se u ovom primeru nazivaju *argument šablona*. Prevodilac na osnovu svake takve upotrebe formira posebnu klasu, zamenjujući *T* datim tipom; taj postupak naziva se *instanciranje šablona* (eng. *instantiation*). Nastale klase `DinamickiNiz<int>` i `DinamickiNiz<double>` dva su različita, nezavisna tipa: objekat jednog ne može se dodeliti objektu drugog, a svaki od njih prolazi uobičajenu proveru tipova. Šablon, dakle, ne odlaže odluku o tipu do izvršavanja programa – sve se razrešava pri prevođenju, pa dobijeni kod nije ni sporiji ni manje bezbedan od ručno napisanog.

Odatle sledi jedno ograničenje koje se lako previdi. *Šablon sme da nad vrednostima tipa T izvodi samo one operacije koje će svaki upotrebljeni tip zaista podržavati*; ako neka od njih nedostaje, greška se prijavljuje pri instanciranju šablona. Naša klasa od tipa *T* zahteva mogućnost stvaranja objekta bez zadate vrednosti i mogućnost dodele¹. Ugrađeni tipovi i uobičajene klase, poput `std::string`, te uslove ispunjavaju.

¹Zbog toga rezervisani deo bloka, prikazan na slici sivom bojom, u našoj implementaciji ne sadrži „prazna mesta“, nego objekte podrazumevane vrednosti. Klasa `std::vector` to izbegava razdvajanjem zauzimanja memorije od stvaranja objekata, čime uklanja i zahtev da tip elemenata ima podrazumevani konstruktor; taj mehanizam prevazilazi okvire ove knjige.

Šabloni u jeziku C++ imaju znatno šire mogućnosti od ovde prikazanih. Nama je dovoljno to što jednu definiciju strukture čine upotrebljivom za bilo koji tip elemenata, pa se time i zadržavamo; sve strukture u narednim glavama definisane su na isti način.

Vratimo se deklaraciji klase. Podatak član i funkcija članica ne mogu da nose isto ime, a i jednim i drugim najbolje pristaje reč *velicina*. Zato imena podataka završavamo donjom crtom, *velicina_* i *kapacitet_*, dok funkcije koje ih čitaju zadržavaju čist oblik. Tu konvenciju zadržavamo kroz celu knjigu.

Funkcije članice mogle bi se definisati unutar same klase, ali ćemo ih, radi preglednosti, u ovoj knjizi pisati odvojeno, ispod deklaracije. Takva definicija mora da kaže kojoj klasi funkcija pripada, i to se postiže operatorom `::`, koji ime povezuje sa oblašću kojoj pripada. Isti operator je već poznat iz zapisa `std::cout`, gde označava ime *cout* iz prostora imena *std*; ovde se upotrebljava analogno, s tim što je oblast klasa umesto prostora imena. Zaglavlje

```
template <typename T>
std::size_t DinamickiNiz<T>::velicina() const
```

čita se, dakle, kao: funkcija *velicina*, koja pripada klasi *DinamickiNiz<T>*, vraća vrednost tipa `std::size_t` i ne menja objekat nad kojim je pozvana (zbog *const* na kraju). Red `template <typename T>` ponavlja se ispred svake takve definicije, jer i ona važi za svaki tip *T*, a ime klase se navodi kao *DinamickiNiz<T>*, pošto klasa bez argumenta šablona nije potpuno ime tipa.

Kod konstruktora i destruktora isti zapis izgleda neobičnije, jer se ime klase javlja dvaput:

```
template <typename T>
DinamickiNiz<T>::DinamickiNiz(std::size_t pocetni_kapacitet)
```

Prvo pojavljivanje označava klasu, drugo je ime same funkcije – konstruktor se uvek zove kao klasa. Argument šablona stoji samo uz prvo, jer se odnosi na klasu. Destruktor se piše na isti način, sa znakom `~` ispred imena funkcije.

Konstruktor formira prazan niz zadatog početnog kapaciteta. Pošto niz u trenutku formiranja nema elemenata, njegova veličina jednaka je nuli.

```
template <typename T>
DinamickiNiz<T>::DinamickiNiz(std::size_t pocetni_kapacitet)
    : elementi(new T[pocetni_kapacitet]),
      velicina_(0),
      kapacitet_(pocetni_kapacitet)
{
}
```

Lista iza znaka `:` jeste *lista inicijalizacije članova* (eng. *member initializer list*): njome članovi objekta dobijaju svoje početne vrednosti pre nego što telo konstruktora počne da se izvršava. U ovom slučaju alokira se blok dovoljan za *pocetni_kapacitet* elemenata, veličina se postavlja na nulu, a kapacitet se pamti. Time je invarijanta iz odeljka 3.1 zadovoljena već u trenutku nastanka objekta – što je opšte svojstvo dobro napisanog konstruktora i razlog zbog kojeg se invarijanta uvek proverava najpre na njemu.

Memorija zauzeta operatorom `new[ ]` mora biti i oslobođena, pa klasa mora da ima *destruktor*:

```
template <typename T>
DinamickiNiz<T>::~~DinamickiNiz()
{
    delete[] elementi;
}
```

Destruktor je funkcija članica koja se poziva automatski, u trenutku kada objekat prestaje da postoji. Njegovo ime jednako je imenu klase ispred kojeg stoji znak ~. Ovim je zaokružen obrazac opisan u glavi 2: resurs se zauzima u konstruktoru, a oslobađa u destrukturu, pa se o njegovom oslobađanju ne mora starati kod koji strukturu koristi. Klasa koja alocira memoriju, a nema destruktor, memoriju gubi.

Veličina i kapacitet su funkcije članice.

```
template <typename T>
std::size_t DinamickiNiz<T>::velicina() const
{
    return velicina_;
}

template <typename T>
std::size_t DinamickiNiz<T>::kapacitet() const
{
    return kapacitet_;
}
```

Oznaka *const* iza zaglavlja znači da funkcija ne menja stanje objekta nad kojim se poziva, pa sme da se pozove i nad konstantnim objektom.

Preostaje osobina zbog koje niz i koristimo – pristup elementu preko indeksa. Želimo da se objekat naše klase u tom pogledu ne razlikuje od niza fiksne dužine:

```
a[3] = 17;
std::cout << a[3];
```

Da bi takav zapis imao smisla, u klasi se definiše funkcija posebnog imena *operator[]*. Time se određuje značenje izraza *a[indeks]* za objekte tipa *DinamickiNiz<T>*. Mogućnost da se postojeći operatori jezika definišu i za korisnički uvedene tipove naziva se *preopterećenje operatora* (eng. *operator overloading*).

```
template <typename T>
T& DinamickiNiz<T>::operator[](std::size_t indeks)
{
    assert(indeks < velicina_);
    return elementi[indeks];
}
```

Bitno je da se vraća *T&*, dakle referenca na element, a ne njegova kopija: samo tako izraz *a[3]* može da stoji sa leve strane dodele. Kada bi se vraćalo *T*, dodela bi menjala kopiju, a element u nizu ostao bi nepromenjen.

Potrebna je i verzija koja se poziva nad konstantnim objektom:

```
template <typename T>
const T& DinamickiNiz<T>::operator[](std::size_t indeks) const
{
    assert(indeks < velicina_);
    return elementi[indeks];
}
```

Dve funkcije imaju isto ime i isti argument, a razlikuju se po tome da li menjaju objekat nad kojim su pozvane. Nad običnim objektom koristi se prva, nad konstantnim druga, koja vraća referencu na konstantu i time onemogućava izmenu elementa.

Obe proveravaju uslov `indeks < velicina_`. Funkcija `assert`, deklarirana u zaglavlju `<cassert>`, prekida izvršavanje programa ako uslov nije ispunjen. Njome ovde iskazujemo pretpostavku pod kojom je operacija uopšte definisana: indeks mora označavati jedan od postojećih elemenata, a ne bilo koje mesto u rezervisanom bloku. Provera nije deo gotovog programa u uobičajenom smislu – prevođenjem sa definisanim makroom `NDEBUG` ona se uklanja – već sredstvo kojim se pretpostavka zapisuje u samom kodu i kojim se njeno kršenje otkriva tokom razvoja.

Provera pomoću funkcije `assert` nije, međutim, mehanizam za obradu grešaka koje se tokom rada programa očekivano javljaju. Za takve situacije C++ ima *izuzetke* (eng. *exceptions*), pomoću naredbi `throw`, `try` i `catch`; standardna biblioteka ih i koristi, pa funkcija `std::vector::at` pri nevažećem indeksu prijavljuje `std::out_of_range`. U ovoj knjizi taj mehanizam ne razvijamo, jer je zasebna tema jezika, a ovde je cilj izraziti pretpostavke pod kojima strukture rade ispravno. Zato se kroz sve naredne glave pretpostavke izražavaju funkcijom `assert`.

Klasa sada omogućava formiranje i uništavanje dinamičkog niza, čitanje veličine i kapaciteta i pristup elementima. Ono što još ne postoji jeste operacija koja veličinu zaista menja – a upravo ona otvara pitanje od kojeg zavisi cela struktura: šta se radi kada je rezervisani kapacitet iscrpljen?

3.3 Rast niza i cena dodavanja elemenata

Osnovna operacija dinamičkog niza jeste dodavanje elementa na kraj. Dok veličina niza nije dostigla kapacitet, operacija je jednostavna: novi element upisuje se na prvu slobodnu poziciju, a veličina se uvećava za jedan.

```
template <typename T>
void DinamickiNiz<T>::dodaj(const T& vrednost)
{
    if (velicina_ < kapacitet_) {
        elementi[velicina_] = vrednost;
        ++velicina_;
        return;
    }

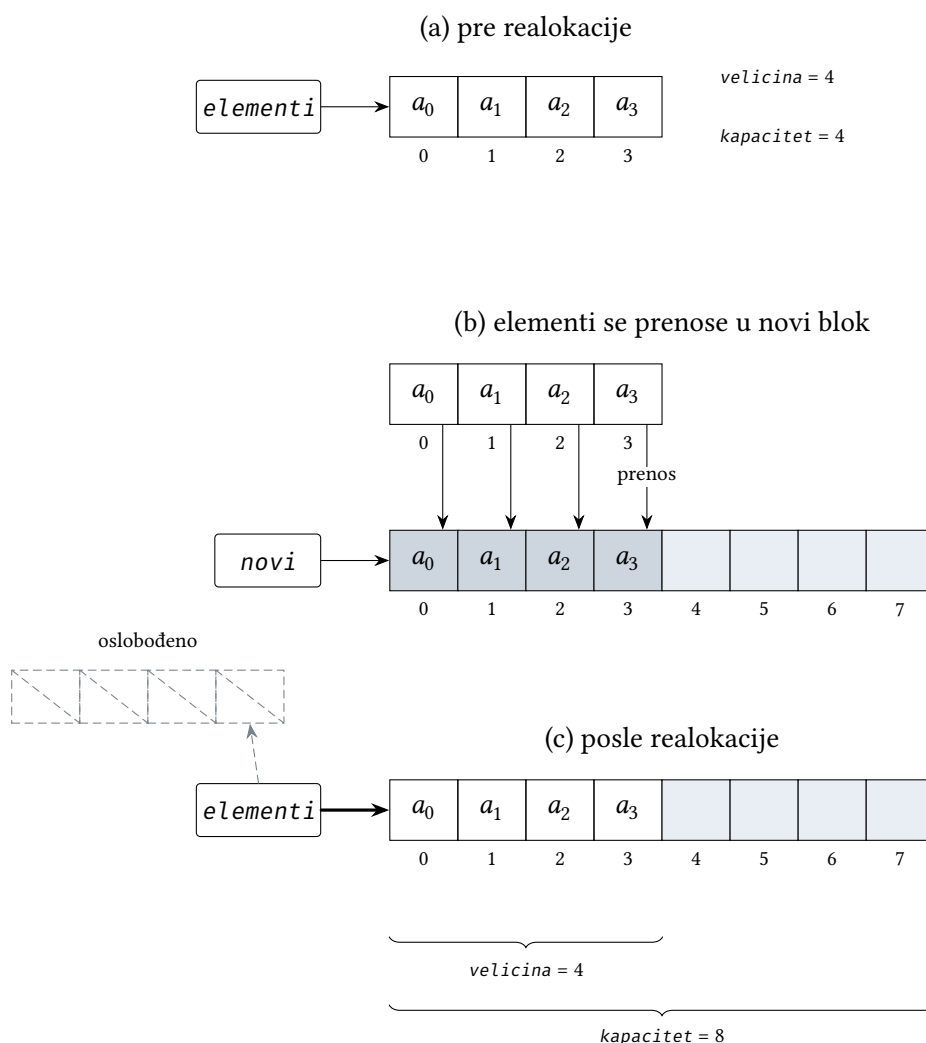
    // rezervisani prostor je iscrpljen
    // ...
}
```

Izvršava se, dakle, konstantan broj koraka, nezavisno od toga koliko elemenata niz već sadrži, pa je dodavanje u ovom slučaju složenosti $O(1)$.

Problem nastaje kada je *velicina_* jednako *kapacitet_*. Postojeći blok je tada popunjen, a nije ga moguće jednostavno proširiti: elementi moraju ostati u uzastopnim memorijskim lokacijama, a ništa ne garantuje da je memorija neposredno iza bloka slobodna – ona je najverovatnije već dodeljena nekom drugom objektu. Umesto proširivanja, mora se obezbediti nov, veći blok i sadržaj preneti u njega. Taj postupak nazivamo *realokacija* (eng. *reallocation*) i sastoji se od pet koraka:

1. alocira se novi, veći blok;
2. postojeći elementi prenose se u novi blok;
3. stari blok se oslobađa;
4. pokazivač *elementi* usmerava se na novi blok;
5. pamti se novi kapacitet.

Slika 3.2 prikazuje taj postupak za niz veličine i kapaciteta četiri, kome se kapacitet udvostručuje.



Slika 3.2: Realokacija dinamičkog niza pri povećanju kapaciteta sa 4 na 8. Elementi se prenose u novi blok, stari blok se oslobađa, a pokazivač se preusmerava na novi blok.

Redosled koraka nije proizvoljan: stari blok sme da se oslobodi tek pošto su svi elementi preneti, a pokazivač sme da se preusmeri tek pošto novi blok postoji. Zamena tih koraka daje viseći pokazivač, o kakvom je bilo reči u glavi 2.

Realokaciju izdvajamo u pomoćnu funkciju, jer je ona unutrašnji posao strukture i korisnik ne mora da zna za postojanje ove funkcije (zapravo, često je poželjno i da nema pristup i da ne može sam da poziva ovu pomoćnu funkciju, pa bi ona uglavnom bila pisana kao *private* funkcija):

```

template <typename T>
void DinamickiNiz<T>::povecaj_kapacitet()
{
    std::size_t novi_kapacitet = 2 * kapacitet_;

    T* novi = new T[novi_kapacitet];

    for (std::size_t i = 0; i < velicina_; ++i)
        novi[i] = elementi[i];

    delete[] elementi;

    elementi = novi;
    kapacitet_ = novi_kapacitet;
}

```

Operacija dodavanja sada se svodi na jedan uslov:

```

template <typename T>
void DinamickiNiz<T>::dodaj(const T& vrednost)
{
    if (velicina_ == kapacitet_)
        povecaj_kapacitet();

    elementi[velicina_] = vrednost;
    ++velicina_;
}

```

Invarijanta je pri tome očuvana i tokom same realokacije: čim se *elementi* i *kapacitet_* promene, oni se odnose na isti, novi blok, a veličina se ne menja jer se broj elemenata prenosom ne povećava.

3.3.1 Cena dodavanja

Ako realokacija nije potrebna, dodavanje košta $O(1)$. Ako jeste, svih n postojećih elemenata mora se preneti u novi blok, pa to jedno dodavanje košta $O(n)$. Na prvi pogled deluje kao da dinamički niz time gubi ono zbog čega smo ga uveli.

Zaključak bi, međutim, bio prenagljen. Realokacija se ne dešava pri svakom dodavanju: pošto se kapacitet uveća, izvestan broj narednih dodavanja ponovo je konstantne cene. Da bismo ocenili stvarnu cenu, nije dovoljno posmatrati najskuplje pojedinačno izvršavanje, nego cenu čitavog niza uzastopnih operacija.

Takva analiza naziva se *amortizovana analiza* (eng. *amortized analysis*). Ako je $T(n)$ ukupna cena najnepovoljnijeg niza od n operacija, amortizovana cena jedne operacije je

$$T(n)/n.$$

Amortizovana analiza nije analiza prosečnog slučaja: ona ne pretpostavlja nikakvu raspodelu ulaza, nego posmatra konkretan niz operacija i ograničava njegovu ukupnu cenu odozgo. Kod dinamičkog niza ta cena zavisi pre svega od strategije po kojoj se bira novi kapacitet, pa razmatramo dve.

Rast za konstantan broj mesta

Prva mogućnost, koju nazivamo *aritmetička strategija*, jeste da se kapacitet pri svakoj realokaciji uveća za unapred izabran broj mesta k , pa kapaciteti redom iznose $k, 2k, 3k, \dots$. Pratimo šta se dešava dok se niz postepeno puni do n elemenata, i brojimo pri tome sve upise u memoriju – kako one kojima se dodaju novi elementi, tako i one kojima se postojeći elementi prenose u novi blok.

Niz počinje sa kapacitetom k , koji se popunjava sa k upisa. Prva realokacija tada obezbeđuje kapacitet $2k$ i prenosi k postojećih elemenata, posle čega sledi k novih upisa. Druga realokacija povećava kapacitet na $3k$ i prenosi $2k$ elemenata, pa opet dolazi k upisa. Treća realokacija daje kapacitet $4k$ i prenosi $3k$ elemenata, i tako redom, sve dok se ne upiše svih n elemenata. Cena se, dakle, smenjuje ovako:

$$\underbrace{k}_{\text{upisi}} + \underbrace{k}_{\text{prenos}} + \underbrace{k}_{\text{upisi}} + \underbrace{2k}_{\text{prenos}} + \underbrace{k}_{\text{upisi}} + \underbrace{3k}_{\text{prenos}} + \dots$$

Pretpostavimo, radi jednostavnosti, da je n deljivo sa k , i označimo $m = n/k$; zaokruživanje u opštem slučaju ne menja red veličine. Upisa novih elemenata ima ukupno n , jer se svaki element upisuje tačno jednom. Realokacija ima $m - 1$, a pri i -toj se prenosi ik elemenata, pa je ukupan broj prenosa

$$k + 2k + \dots + (m-1)k = k \sum_{i=1}^{m-1} i = k \cdot \frac{(m-1)m}{2} = \frac{n^2}{2k} - \frac{n}{2}.$$

Ukupna cena dodavanja n elemenata je zbir upisa i prenosa,

$$T(n) = \frac{n^2}{2k} + \frac{n}{2} = O(n^2),$$

a amortizovana cena jednog dodavanja

$$\frac{T(n)}{n} = \frac{n}{2k} + \frac{1}{2} = O(n).$$

Aritmetička strategija, dakle, ne daje ništa bolje od alokacije tačne dužine pri svakom dodavanju. Vidi se i zašto: koliko god k bilo veliko, ono u izrazu za ukupnu cenu stoji kao delilac, pa je smanjuje konstantan broj puta, ali ne menja njen red. Veći korak samo pomera trenutak u kome cena postane neprihvatljiva, kao što je i veći statički niz iz odeljka 3 samo pomerao trenutak u kome ponestane mesta.

Rast za konstantan činiac

Druga mogućnost, *geometrijska strategija*, jeste da se kapacitet pri svakoj realokaciji pomnoži konstantnim činiocem $q > 1$. Ako je početni kapacitet m_0 , kapaciteti redom iznose $m_0, qm_0, q^2m_0, \dots$. Brojimo upise na isti način kao kod aritmetičke strategije.

Posle početne alokacije upisuje se m_0 elemenata. Prva realokacija podiže kapacitet na qm_0 i prenosi m_0 postojećih elemenata, posle čega se upisuje još $qm_0 - m_0$ novih. Druga realokacija daje kapacitet q^2m_0 i prenosi qm_0 elemenata, pa se upisuje još $q^2m_0 - qm_0$ novih, i tako redom. Zbir svih upisa posle r realokacija je zato

$$m_0 + [m_0 + (qm_0 - m_0)] + [qm_0 + (q^2m_0 - qm_0)] + \dots = m_0 + qm_0 + q^2m_0 + \dots + q^r m_0,$$

jer se u svakoj uglastoj zagradi prenos i upisi sabiraju u tačno onoliko koliko iznosi novi kapacitet. Suma geometrijskog niza daje

$$m_0 (1 + q + q^2 + \dots + q^r) = m_0 \cdot \frac{q^{r+1} - 1}{q - 1}.$$

Pretpostavimo da je niz posle poslednje realokacije popunjen, odnosno da je $n = m_0 q^r$. Zamenom se dobija ukupna cena dodavanja n elemenata:

$$T(n) = m_0 \cdot \frac{q \cdot n / m_0 - 1}{q - 1} = \frac{qn - m_0}{q - 1} = O(n),$$

pa je amortizovana cena jednog dodavanja $T(n)/n = O(1)$.

Konstanta uz n iznosi $q/(q - 1)$ i opada kako q raste: pri udvostručavanju, $q = 2$, ona je jednaka dva, što znači da se po dodatom elementu u proseku obavlja manje od dva upisa. To se vidi i bez opšte formule. Ako kapaciteti redom iznose 1, 2, 4, 8, ..., tokom svih realokacija prenosi se najviše $1 + 2 + 4 + \dots + n/2 < n$ elemenata, čemu se dodaje n upisa novih elemenata.

Iako pojedinačno dodavanje povremeno košta $O(n)$, skupa dodavanja javljaju se dovoljno retko da cena niza od n dodavanja ostaje linearna. Zato se kaže da dodavanje na kraj dinamičkog niza ima *amortizovanu složenost* $O(1)$. To je i prvi primer razlike koja se u knjizi više puta javlja: najgori pojedinačni slučaj i cena po operaciji u dužem nizu operacija ne moraju biti istog reda.

Geometrijska strategija, međutim, nije besplatna. Veći činilac znači ređe realokacije i manju konstantu $q/(q - 1)$, ali i više neiskorišćene memorije: pri udvostručavanju je neposredno posle rasta popunjena tek polovina bloka. Manji činilac štedi memoriju, a plaća češćim realokacijama i većom konstantom. Izbor činioca zato nije pitanje složenosti – ona je $O(1)$ za svako $q > 1$ – nego kompromis između utrošenog vremena i utrošene memorije.

Standardna biblioteka počiva na istoj ideji. Standard ne propisuje činilac kojim kapacitet raste, nego ga prepušta implementaciji; ono što garantuje jeste da dodavanje na kraj ima amortizovanu konstantnu cenu. Različite implementacije zato biraju različite vrednosti, najčešće 2 ili 1,5.

3.4 Umetanje i brisanje

Dodavanje na kraj niza jeftino je pre svega zato što postojeći elementi pri tome ostaju na svojim mestima. Umetanje unutar niza tu pogodnost nema: pozicije su određene indeksima, pa novi element može da zauzme mesto samo ako se ono najpre oslobodi.

Neka niz sadrži elemente a_0, a_1, a_2, a_3, a_4 i neka na poziciju 2 treba umetnuti element x . Posle umetanja niz treba da glasi $a_0, a_1, x, a_2, a_3, a_4$, pa se svi elementi počev od a_2 moraju pomeriti za jedno mesto udesno, kao na slici 3.3.

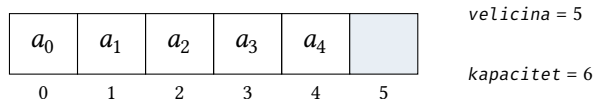
Smer pomeranja nije proizvoljan: kreće se od poslednjeg elementa ka mestu umetanja. Kada bi se elementi pomerali redom sleva nadesno, prvo bi upisivanje prepisalo element koji tek treba pomeriti, pa bi se ista vrednost umnožila kroz ostatak niza.

```
template <typename T>
void DinamickiNiz<T>::umetni(std::size_t indeks, const T& vrednost)
{
    assert(indeks <= velicina_);

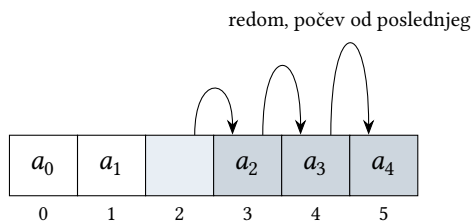
    if (velicina_ == kapacitet_)
        povecaj_kapacitet();

    for (std::size_t i = velicina_; i > indeks; --i)
```

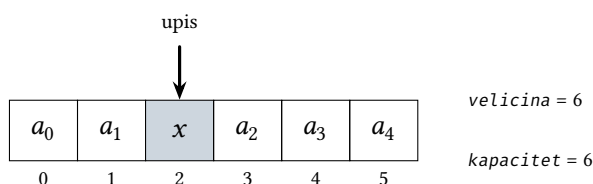
(a) pre umetanja



(b) elementi se pomeraju udesno



(c) posle umetanja



Slika 3.3: Umetanje elementa na poziciju 2. Elementi od pozicije umetanja do kraja niza pomeraju se za jedno mesto udesno, počev od poslednjeg, a zatim se novi element upisuje na oslobođeno mesto.

```

    elementi[i] = elementi[i - 1];

    elementi[indeks] = vrednost;
    ++velicina_;
}

```

Uslov $indeks \leq velicina_$ blaži je nego kod pristupa elementu, gde smo zahtevali $indeks < velicina_$. Razlog je što je umetanje na poziciju $velicina_$ smisljeno: tada se nijedan element ne pomera, a operacija se svodi na dodavanje na kraj.

Pri umetanju na poziciju $indeks$ pomera se $velicina_ - indeks$ elemenata. U najgorem slučaju, kada se umeće na početak, pomeraju se svi, pa je složenost umetanja $O(n)$. Ta cena ne potiče od moguće realokacije – realokacije su, kao što smo videli u odeljku 3.3, dovoljno retke da im je amortizovani doprinos konstantan – nego od samog pomeranja, koje se obavlja i kada slobodnog kapaciteta ima napretek.

Brisanje elementa iz unutrašnjosti niza susreće se sa istom preprekom, samo u suprotnom smeru. Brisanjem elementa na poziciji 2 iz niza a_0, a_1, a_2, a_3, a_4 dobija se a_0, a_1, a_3, a_4 , što znači da se svaki element iza obrisanog mora pomeriti za jedno mesto ulevo. Pomeranje se ovoga puta obavlja od mesta brisanja ka kraju niza, iz istog razloga iz kojeg je kod umetanja išlo obrnuto.

```

template <typename T>
void DinamickiNiz<T>::obrisi(std::size_t indeks)
{
    assert(indeks < velicina_);

    for (std::size_t i = indeks; i + 1 < velicina_; ++i)
        elementi[i] = elementi[i + 1];
}

```

```
--velicina_;
}
```

Uslov petlje napisan je kao $i + 1 < \text{velicina_}$, a ne kao $i < \text{velicina_} - 1$, iako su ta dva izraza matematički istovetna. Tip `std::size_t` je bez znaka, pa se oduzimanje jedinice od nule ne završava vrednošću -1 , nego najvećim brojem koji taj tip može da predstavi; petlja bi u tom slučaju krenula da čita daleko izvan niza. Ovde je istina da naredba `assert` unapred isključuje praznu strukturu, ali je navika da se oduzimanje izbegava korisna.

Umetanje i brisanje ostavljaju kapacitet nepromenjenim. Posle brisanja veličina je manja, dok pokazivač i kapacitet ostaju isti, pa je invarijanta očuvana, a oslobođeno mesto ostaje rezervisano i biće iskorišćeno pri narednom dodavanju. Nizu, dakle, ne smanjujemo kapacitet automatski; to bi značilo da bi naizmenično dodavanje i brisanje jednog elementa oko granice kapaciteta izazivalo realokaciju pri svakoj operaciji i time pokvarilo amortizovanu ocenu iz prethodnog odeljka.

Poseban je slučaj brisanja poslednjeg elementa: nijedan element se ne pomera, pa se operacija svodi na umanjeње veličine i ima složenost $O(1)$.

Time se zaokružuje slika o ceni operacija dinamičkog niza. Pristup elementu po indeksu i dodavanje na kraj jeftini su – prvi u najgorem, drugi u amortizovanom smislu – dok umetanje i brisanje unutar niza u najgorem slučaju koštaju $O(n)$. Taj nesrazmer nije posledica implementacije koju smo izabrali, nego same odluke da elementi budu smešteni u uzastopnim memorijskim lokacijama. Struktura kod koje je taj odnos obrnut – umetanje jeftino, pristup po indeksu skup – predmet je glave 4.

3.5 Kopiranje niza

Klasa koja poseduje dinamički alociranu memoriju traži posebnu pažnju pri kopiranju. Razlog je što kopiranje član po član, koje prevodilac obezbeđuje sam od sebe, nije dovoljno kada je jedan od članova pokazivač na memoriju u vlasništvu objekta.

Posmatrajmo dva objekta:

```
DinamickiNiz<int> a(4);
a.dodaj(10);
a.dodaj(20);
a.dodaj(30);

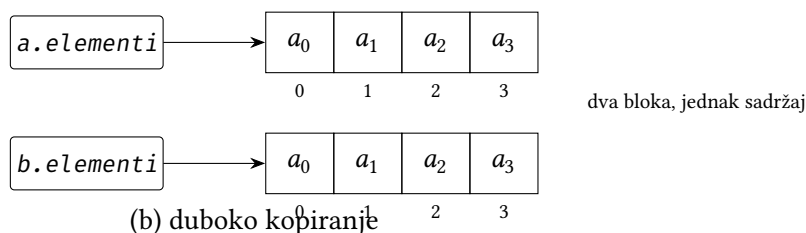
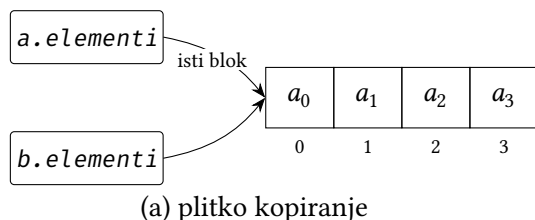
DinamickiNiz<int> b = a;
```

Ako sami ne propišemo kako se kopiranje obavlja, prevodilac će novi objekat obrazovati tako što će svaki član objekta *a* prepisati u odgovarajući član objekta *b*. Za *velicina_* i *kapacitet_* to je upravo ono što želimo. Kod pokazivača *elementi* nastaje problem: prepisuje se adresa, pa oba objekta ukazuju na isti memorijski blok, kao na levoj strani slike 3.4.

Kopiranje koje prepisuje samo vrednost pokazivača naziva se *plitko kopiranje* (eng. *shallow copy*) i za klasu *DinamickiNiz* je pogrešno iz dva razloga.

Prvi je što objekti prestaju da budu nezavisni. Dodela `b[0] = 42` promenila bi i prvi element objekta *a*, jer oba čitaju iz istog bloka – a ništa u zapisu te naredbe ne nagoveštava da se menja još nešto osim objekta *b*.

Drugi razlog je teže uočiti. Kada oba objekta prestanu da postoje, svaki od njih izvršiće `delete[] elementi` nad istom adresom. Prvi destruktork oslobodiće blok, a drugi će pokušati da oslobodi memoriju koja više nije zauzeta; ishod je nedefinisano ponašanje, o kome je bilo reči u glavi 2.



Slika 3.4: Plitko i duboko kopiranje dinamičkog niza. Pri plitkom kopiranju oba objekta ukazuju na isti blok; pri dubokom svaki objekat ima sopstveni.

Da bi kopija bila nezavisan objekat, mora se izvršiti *duboko kopiranje* (eng. *deep copy*): novi objekat dobija sopstveni memorijski blok, u koji se prepisuju elementi izvornog niza. Ponašanje pri obrazovanju novog objekta na osnovu postojećeg propisuje se *konstruktorom kopije* (eng. *copy constructor*), koji se u klasi deklariraše kao

```
DinamickiNiz(const DinamickiNiz& drugi);
```

a definiše ovako:

```
template <typename T>
DinamickiNiz<T>::DinamickiNiz(const DinamickiNiz<T>& drugi)
    : elementi(new T[drugi.kapacitet_]),
      velicina_(drugi.velicina_),
      kapacitet_(drugi.kapacitet_)
{
    for (std::size_t i = 0; i < velicina_; ++i)
        elementi[i] = drugi.elementi[i];
}
```

Parametar je konstantna referenca: referenca zato što bi prenos po vrednosti zahtevao kopiju objekta koji tek treba kopirati, a *const* zato što se izvorni objekat ne menja. Konstruktor kopije sme da čita privatne članove objekta *drugi*, jer pristup privatnim članovima imaju sve funkcije članice klase, a ne samo one pozvane nad datim objektom.

Posle ovoga naredba `DinamickiNiz<int> b = a;` obrazuje objekat *b* sa sopstvenim blokom i u njega prepisuje elemente objekta *a*. Dva objekta imaju jednak sadržaj, jednaku veličinu i jednak kapacitet, ali različite blokove – to je desna strana slike 3.4.

Kopiranje pri nastanku objekta treba, međutim, razlikovati od dodele objektu koji već postoji. U nizu naredbi

```
DinamickiNiz<int> a(4);
DinamickiNiz<int> b(20);

b = a;
```

objekat *b* postoji od ranije i već poseduje sopstveni blok, pa se konstruktor ne poziva. Umesto njega poziva se *operator dodele* (eng. *assignment operator*), koji se definiše kao funkcija članica posebnog imena *operator=*:

```
template <typename T>
DinamickiNiz<T>& DinamickiNiz<T>::operator=(const DinamickiNiz<T>& drugi)
{
    if (this == &drugi)
        return *this;

    T* novi = new T[drugi.kapacitet_];

    for (std::size_t i = 0; i < drugi.velicina_; ++i)
        novi[i] = drugi.elementi[i];

    delete[] elementi;

    elementi = novi;
    velicina_ = drugi.velicina_;
    kapacitet_ = drugi.kapacitet_;

    return *this;
}
```

Operator dodele složeniji je od konstruktora kopije upravo zato što levi operand zatiče u posedu blok koji mora biti oslobođen. Redosled koraka je i ovde bitan, iz istog razloga kao pri realokaciji: novi blok se najpre obezbedi i popuni, pa se tek onda stari oslobađa. Kada bi se *delete[]* izvršio na početku, a alokacija novog bloka zatim ne bi uspeła, objekat bi ostao u stanju koje krši invarijantu.

Poseban slučaj jeste dodela objekta samom sebi, *a = a*. Ona u ovom obliku deluje kao nešto što niko ne piše, ali se lako javlja posredno. Bez provere na početku funkcije, objekat bi oslobodio sopstveni blok, a zatim iz njega pokušao da prepíše elemente. Pokazivač *this* u funkciji članici ukazuje na objekat nad kojim je funkcija pozvana, pa poređenje *this == &drugi* upravo utvrđuje da li su levi i desni operand isti objekat.

Funkcija na kraju vraća **this*, odnosno referencu na objekat kome je dodela izvršena, čime se zadržava uobičajeno ponašanje operatora *=* u jeziku C++ i ostaje moguć zapis *a = b = c*.

Klasa dinamičkog niza tako je dobila tri funkcije koje zajedno upravljaju vlasništvom nad memorijskim blokom. Destruktor blok oslobađa, konstruktor kopije obrazuje nezavisnu kopiju, a operator dodele zamenjuje blok objekta koji već postoji. Njihova povezanost nije slučajna: kada klasa neposredno upravlja resursom, potreba za bilo kojom od te tri funkcije po pravilu znači da su potrebne i preostale dve, jer sve tri odgovaraju na isto pitanje – ko je vlasnik resursa i kada ga oslobađa.²

Ponekad, međutim, nije poželjno da se objekat uopšte može kopirati. To se postiže zabranom:

²U literaturi se ovo načelo naziva *pravilom trojke* (eng. *rule of three*): ako je klasi potrebna bilo koja od tri navedene funkcije, potrebne su joj sve tri. Savremeni C++ uvodi i dve funkcije koje resurs ne kopiraju nego ga premeštaju iz objekta koji se više neće koristiti, čime, na primer, niz vraćen iz funkcije može da se preuzme bez kopiranja elemenata.

```
Klasa(const Klasa&) = delete;
Klasa& operator=(const Klasa&) = delete;
```

Oznakom `= delete` prevodiocu se nalaže da te dve funkcije ne generiše. Svaki pokušaj kopiranja takvog objekta prijavljuje se kao greška pri prevođenju.

3.6 Dinamički niz u standardnoj biblioteci

Klasa `std::vector` je standardna implementacija dinamičkog niza. Sve što je u ovoj glavi izloženo odnosi se, dakle, i na nju: razdvajanje veličine od kapaciteta, realokacija pri rastu, pomeranje elemenata pri umetanju i brisanju. U ovoj glavi opisan je odnos prikazane implementacije i implementacije iz standardne biblioteke:

<i>DinamickiNiz<T></i>	<i>std::vector<T></i>
<i>velicina()</i>	<i>size()</i>
<i>kapacitet()</i>	<i>capacity()</i>
<i>dodaj(x)</i>	<i>push_back(x)</i>
<i>umetni(i, x)</i>	<i>insert(...)</i>
<i>obrisi(i)</i>	<i>erase(...)</i>
<i>operator[]</i> uz <i>assert</i>	<i>operator[], at(i)</i>

Postoje i razlike. Pozicija umetanja i brisanja kod vektora se ne zadaje indeksom, nego posebnim objektom koji označava mesto u kontejneru; taj pojam uvodimo u glavi 4, gde postaje neophodan. Za pristup elementu vektor nudi dve mogućnosti: `operator[]`, koji, kao i kod niza fiksne dužine, ne proverava indeks, i funkciju `at`, koja proveru vrši i pri nevažjećem indeksu prijavljuje izuzetak `std::out_of_range`. Time standardna biblioteka nudi oba pristupa pomenuta u odeljku 3.2: `operator[]` polazi od toga da je ispravnost indeksa obaveza onoga ko klasu koristi, dok `at` grešku prijavljuje izuzetkom. U kodu u kome indeks potiče iz spoljašnjeg izvora, `at` je sigurniji izbor, jer grešku prijavljuje odmah; unutar petlje koja indeks sama proverava, dodatna provera ne donosi ništa novo.

Složenosti operacija sažete su u sledećoj tabeli i važe podjednako za obe implementacije, pošto proističu iz reprezentacije, a ne iz načina na koji je ona programirana.

Operacija	Složenost
pristup elementu po indeksu	$O(1)$
izmena elementa po indeksu	$O(1)$
dobijanje broja elemenata	$O(1)$
dodavanje na kraj	amortizovano $O(1)$
uklanjanje sa kraja	$O(1)$
umetanje na proizvoljnu poziciju	$O(n)$
brisanje sa proizvoljne pozicije	$O(n)$
pretraga u neuređenom nizu	$O(n)$

Dinamički niz je zato prirodan izbor kada se elementima pristupa po indeksu ili se obilaze redom, a kolekcija raste na kraju i njena veličina nije unapred poznata. Nije dobar izbor kada se često umeće

Isti uslov tada se proširuje na svih pet – otuda i naziv *pravilo petorke*. Premeštanje izlazi iz okvira ove knjige, u kojoj se zadržavamo na tri navedene funkcije.

ili briše na početku ili u sredini velike kolekcije: umetanje na početak pomera svih n elemenata, pa algoritam kome je to glavna operacija plaća linearnu cenu u svakom koraku. Odluku o strukturi ne treba, dakle, donositi prema tome koje operacije struktura *podržava* – vektor podržava i umetanje na početak – nego prema tome koliko koštaju one koje se izvršavaju često.

4

Povezane liste

Dinamički niz, kojim se bavila glava 3, čuva elemente u jednom neprekidnom bloku memorije. Iz te odluke proističe njegova najveća prednost – pristup elementu sa zadatim indeksom u vremenu $O(1)$ – ali i njegova ograničenja. Umetanje ili brisanje na početku ili u sredini niza zahteva pomeranje svih elemenata iza te pozicije, a kada se kapacitet popuni, realokacija traži nov, veći blok i kopiranje celog sadržaja.

Povezana lista (eng. *linked list*) polazi od suprotne odluke: odustaje od neprekidnosti. Svaki element čuva se u zasebno alociranom delu memorije, a redosled elemenata ne određuje njihov položaj u memoriji, već pokazivači koji vode od jednog elementa do drugog. Zahvaljujući tome, umetanje i brisanje na poznatom mestu svode se na izmenu nekoliko pokazivača, bez pomeranja ostalih elemenata. Cena je gubitak direktnog pristupa: do elementa se stiže samo praćenjem pokazivača od početka liste.

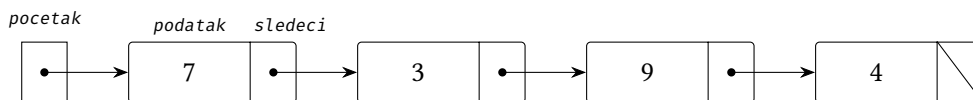
U ovoj glavi najpre detaljno razmatramo jednostruko povezanu listu i njene operacije, a zatim je poredimo sa dinamičkim nizom, kako po složenosti, tako i po ponašanju na stvarnom računaru. Potom prikazujemo kružnu i dvostruko povezanu listu, odgovarajuće kontejnere standardne biblioteke i, na kraju, situacije u kojima lista jeste, odnosno nije, dobar izbor.

4.1 Jednostruko povezana lista

Jednostruko povezana lista (eng. *singly linked list*) sastoji se od čvorova (eng. *node*). Svaki čvor sadrži podatak i pokazivač na sledeći čvor liste. Ti pokazivači povezuju čvorove u celinu, i to samo u jednom smeru: iz čvora se može preći na njegovog sledbenika, ali ne i na prethodnika. Poslednji čvor nema sledbenika, pa njegov pokazivač ima vrednost *nullptr*.

Čvorovi se alociraju pojedinačno, u dinamičkoj memoriji, pa se mogu nalaziti na međusobno udaljenim adresama (slika 4.1).

(a) neprazna lista



(b) prazna lista



Slika 4.1: Jednostruko povezana lista sa četiri elementa. Pokazivač poslednjeg čvora ima vrednost *nullptr*.

Listi se pristupa preko pokazivača na njen prvi čvor, koji nazivamo *početak* liste (eng. *head*). Od početka se, praćenjem pokazivača, stiže do svakog čvora, pa je taj pokazivač jedino što treba čuvati da bi cela lista bila dostupna. Zato se on mora pažljivo održavati: ako se njegova vrednost izgubi, čvorovi ostaju u memoriji, ali do njih više nije moguće doći. To je upravo curenje memorije, opisano u odeljku 2.5. Prazna lista nema nijedan čvor, pa njen početak ima vrednost *nullptr*.

4.2 Čvor liste

Čvor liste čuva dve stvari: podatak i pokazivač na sledeći čvor. Opisujemo ga strukturom:

```
template <typename T>
struct Cvor {
    T podatak;
    Cvor* sledeci;

    Cvor(const T& p, Cvor* s) : podatak(p), sledeci(s) {}
};
```

Na prvi pogled je neobično što struktura *Cvor* u svojoj definiciji pominje samu sebe. Čvor zaista ne može da sadrži drugi čvor, jer bi tada i taj drugi sadržao treći, i tako bez kraja. Polje *sledeci*, međutim, nije čvor, već pokazivač na čvor. Svi pokazivači zauzimaju jednako memorije, pa prevodilac zna veličinu polja *sledeci* i pre nego što je definicija strukture završena.

Čvorovi se prave jedan po jedan, operatorom *new*, i tek zatim povezuju sa ostatkom liste:

```
Cvor<int>* novi = new Cvor<int>(7, nullptr);
```

Novi čvor sadrži podatak 7 i još nema sledbenika. Podatak se pri tome *kopira* u čvor. Čvor liste čiji su elementi tipa *std::string* dobija, na primer, sopstvenu kopiju niske, pa kasnija izmena originala ne utiče na sadržaj liste.

4.2.1 Klasa *Lista*

Korisnik liste ne treba sam da pravi, povezuje i oslobađa čvorove. Kao i kod dinamičkog niza, taj posao preuzima klasa. Od liste očekujemo umetanje i brisanje na početku, umetanje i brisanje iza zadatog čvora, pretragu, kao i podatak o broju elemenata.

```
template <typename T>
class Lista {
public:
    Lista() : pocetak_(nullptr), velicina_(0) {}
    ~Lista();

    Lista(const Lista&) = delete;
    Lista& operator=(const Lista&) = delete;

    bool prazna() const { return pocetak_ == nullptr; }
    std::size_t velicina() const { return velicina_; }
    Cvor<T>* pocetak() const { return pocetak_; }
```

```

void dodaj_na_pocetak(const T& x);
void umetni_iza(Cvor<T>* c, const T& x);
void umetni_ispred(Cvor<T>* c, const T& x);
void umetni_sortirano(const T& x);

void obrisi_sa_pocetka();
void obrisi_iza(Cvor<T>* c);
bool obrisi(const T& x);
void isprazni();

Cvor<T>* pronadji(const T& x) const;

private:
    Cvor<T>* pocetak_;
    std::size_t velicina_;
};

```

Klasa ima samo dva člana. Pokazivač *pocetak_* pokazuje na prvi čvor, a *velicina_* čuva broj elemenata, da ih ne bismo prebrojavali pri svakom pozivu funkcije *velicina()*. Konstruktor pravi praznu listu.

Lista je vlasnik svojih čvorova: ona ih alocira, pa ih ona mora i osloboditi. To radi destruktor, prikazan u odeljku 4.4. Kopiranje liste je zabranjeno. Podrazumevano kopiranje prepisalo bi samo pokazivač *pocetak_*, pa bi dve liste delile iste čvorove i obe bi pokušale da ih oslobode. Taj problem i njegovo rešenje, duboko kopiranje, opisani su u odeljku 3.5.

Sve operacije moraju da čuvaju sledeće svojstvo.

Invarijanta liste. Ako se krene od *pocetak_* i prate pokazivači *sledeci*, posle tačno *velicina_* čvorova stiže se do *nullptr*. Svaki čvor na tom putu alocirala je lista i nijedan se ne pojavljuje dva puta.

Invarijanta, između ostalog, kaže da je u praznoj listi *pocetak_* jednak *nullptr*, kao i da u listi nema ciklusa: praćenjem pokazivača uvek se stiže do kraja.

4.3 Obilazak liste

Gotovo svaka operacija nad listom zasniva se na *obilasku*: krećemo od početka i pratimo pokazivače *sledeci* dok ne stignemo do *nullptr*. Obilazak se uvek piše na isti način:

```

for (Cvor<T>* p = pocetak_; p != nullptr; p = p->sledeci) {
    // obrada čvora na koji pokazuje p
}

```

Pokazivač *p* redom pokazuje na svaki čvor liste. Naredba *p = p->sledeci* ima istu ulogu kao *++i* u obilasku niza: prelazi na sledeći element. Obilazak cele liste je složenosti $O(n)$.

Lista se može obići samo unapred. Elementi se u obrnutom redosledu mogu obraditi rekurzijom, tako što funkcija prvo pozove samu sebe za ostatak liste, pa tek onda obradi tekući čvor. Takvo rešenje je kratko, ali ima skrivenu cenu.

Kao što smo videli u odeljku 2.4, svaki poziv funkcije dobija na programskom steku sopstveni prostor za lokalne promenljive, i taj prostor se oslobađa tek kada se poziv završi. Rekurzivna funkcija nad listom od n elemenata pravi n ugnežđenih poziva, a nijedan se ne završava pre nego što se dođe do kraja liste. Zato su u jednom trenutku na steku svih n poziva odjednom.

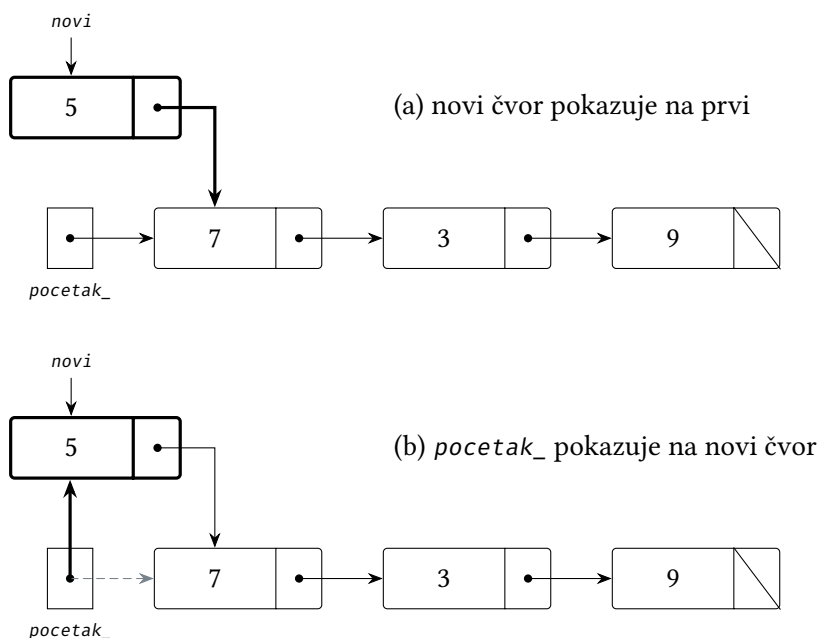
Stek je, međutim, mali: obično ima svega nekoliko megabajta, dok čvorovi liste u dinamičkoj memoriji mogu zauzeti mnogo više. Kada se stek popuni, program se prekida, najčešće uz poruku *Segmentation fault*. Na uobičajenom računaru to se dešava već kod liste od nekoliko stotina hiljada elemenata, dakle kod liste koju program lako napravi. Iterativni obilazak nema taj problem, jer koristi samo jedan poziv i jedan pokazivač, ma koliko lista bila duga. Zato operacije nad listama pišemo iterativno.

4.4 Umetanje i brisanje

4.4.1 Umetanje na početak

Novi element najlakše je dodati na početak liste. Potrebna su dva koraka: novi čvor treba da pokaže na dosadašnji prvi čvor, a zatim *pocetak_* treba da pokaže na novi čvor (slika 4.2).

```
template <typename T>
void Lista<T>::dodaj_na_pocetak(const T& x)
{
    Cvor<T>* novi = new Cvor<T>(x, pocetak_);
    pocetak_ = novi;
    ++velicina_;
}
```



Slika 4.2: Umetanje elementa 5 na početak liste: (a) novi čvor pokazuje na dosadašnji prvi čvor; (b) *pocetak_* pokazuje na novi čvor.

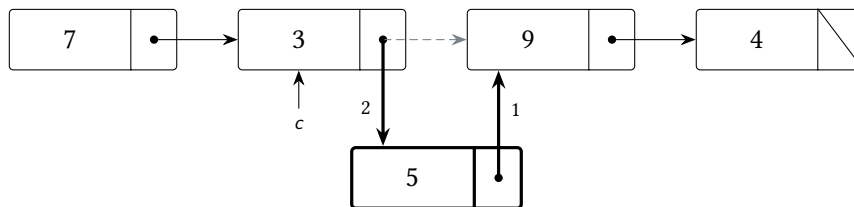
Prvi korak obavlja konstruktor čvora, koji polje *sledeci* postavlja na trenutnu vrednost

pocetak_. Funkcija radi ispravno i kada je lista prazna: tada je *pocetak_* jednak *nullptr*, pa novi čvor postaje i prvi i poslednji. Broj koraka ne zavisi od dužine liste, pa je složenost $O(1)$.¹

4.4.2 Umetanje iza datog čvora

Ako je poznat pokazivač *c* na neki čvor liste, novi element se iza njega umeće na isti način. Novi čvor prvo pokaže na sledbenika čvora *c*, a zatim čvor *c* pokaže na novi čvor (slika 4.3).

```
template <typename T>
void Lista<T>::umetni_iza(Cvor<T>* c, const T& x)
{
    Cvor<T>* novi = new Cvor<T>(x, c->sledeci);
    c->sledeci = novi;
    ++velicina_;
}
```



Slika 4.3: Umetanje elementa 5 iza čvora *c*. Isprekidana strelica prikazuje vezu koja se zamenjuje.

Redosled ova dva koraka je važan. Ako bismo prvo izmenili *c->sledeci*, izgubili bismo adresu ostatka liste, jer je ona zapisana jedino u tom polju. Funkcija pretpostavlja da *c* pokazuje na čvor liste; poziv sa *nullptr* nije dozvoljen. I ova operacija je složenosti $O(1)$, i to je glavna prednost liste nad nizom: nijedan postojeći element se ne pomera.

Umetanje *ispred* čvora *c* je teže, jer iz čvora ne možemo doći do njegovog prethodnika. Problem se može zaobići: novi čvor umetnemo iza čvora *c*, u njega prepisemo podatak iz čvora *c*, a u čvor *c* upišemo novi podatak.

```
template <typename T>
void Lista<T>::umetni_ispred(Cvor<T>* c, const T& x)
{
    umetni_iza(c, c->podatak);
    c->podatak = x;
}
```

Redosled elemenata je onaj koji smo želeli, a složenost je i dalje $O(1)$. Cena je to što se jedan podatak kopira.

Dodavanje iza poslednjeg elementa zahteva obilazak cele liste, pa je složenosti $O(n)$. Ako je ta operacija česta, klasa treba da čuva i pokazivač na poslednji čvor, i tada je dodavanje složenosti $O(1)$. Takvu listu koristimo u glavi ??, za implementaciju reda.

¹Ako nema dovoljno memorije, izraz *new* prijavljuje izuzetak *std::bad_alloc*, kao što je rečeno u odeljku 2.4. Zato funkcije za umetanje ne proveravaju uspeh alokacije.

4.4.3 Brisanje

Prvi element se briše u tri koraka: zapamtimo adresu prvog čvora, pomerimo *pocetak_* na drugi čvor i oslobodimo zapamćeni čvor.

```
template <typename T>
void Lista<T>::obrisi_sa_pocetka()
{
    assert(!prazna());
    Cvor<T>* stari = pocetak_;
    pocetak_ = stari->sledeci;
    delete stari;
    --velicina_;
}
```

Pomoćni pokazivač *stari* je neophodan. Kada bismo prvo izvršili *delete pocetak_*, više ne bismo smeli da čitamo polje *sledeci* oslobođenog čvora, pa bi ostatak liste bio izgubljen. Šta raditi ako je lista prazna? Funkcija bi mogla da tada jednostavno ne uradi ništa, ali bi time sakrila grešku: ko briše iz prazne liste, pogrešno pretpostavlja da u njoj ima elemenata, pa bi program nastavio rad sa tom pogrešnom pretpostavkom. Zato je neprazna lista preduslov funkcije, a njegovo kršenje otkriva *assert*, kao i kod dinamičkog niza u odeljku 3.2. Drugačije je kod funkcije *obrisi(x)* iz odeljka 4.5: tamo je to što traženi element ne postoji sasvim običan ishod, pa funkcija vraća *false*.

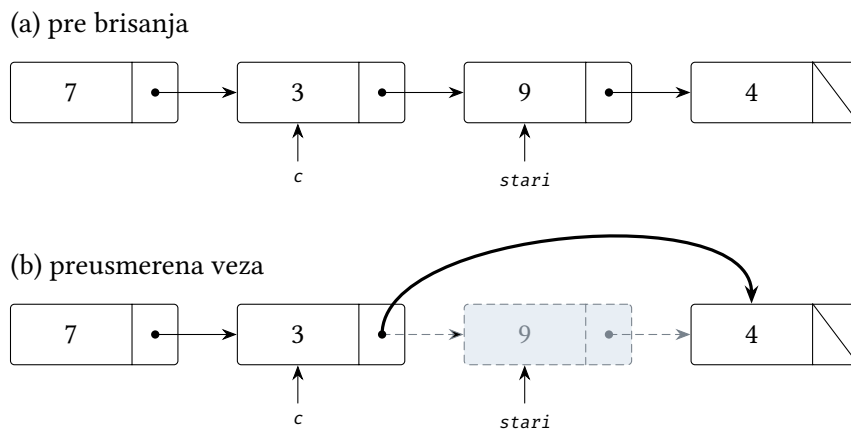
Brisanje iza zadatog čvora *c* izvodi se na isti način, samo se umesto *pocetak_* menja polje *c->sledeci*: čvor *c* „preskače“ svog sledbenika (slika 4.4).

```
template <typename T>
void Lista<T>::obrisi_iza(Cvor<T>* c)
{
    Cvor<T>* stari = c->sledeci;
    assert(stari != nullptr);
    c->sledeci = stari->sledeci;
    delete stari;
    --velicina_;
}
```

Obe operacije su složenosti $O(1)$. Obratimo pažnju na to šta nije moguće: čvor ne možemo obrišati ako znamo samo njegovu adresu, jer treba izmeniti polje *sledeci* njegovog prethodnika. Iz istog razloga je brisanje poslednjeg elementa složenosti $O(n)$, čak i kada klasa čuva pokazivač na poslednji čvor. Ovo ograničenje otklanja dvostruko povezana lista (odeljak 4.9).

Kada lista prestane da postoji, svi njeni čvorovi moraju biti oslobođeni. Najjednostavnije je brisati prvi element sve dok lista ne postane prazna:

```
template <typename T>
void Lista<T>::isprazni()
{
    while (!prazna())
        obrisi_sa_pocetka();
}
```



Slika 4.4: Brisanje elementa iza čvora *c*: (a) pokazivač *stari* pamti čvor koji se briše; (b) čvor *c* pokazuje na sledbenika obrisanog čvora, a obrisani čvor se oslobađa.

```
template <typename T>
Lista<T>::~Lista()
{
    isprazni();
}
```

Pražnjenje je složenosti $O(n)$, jer se svaki čvor oslobađa posebno.

4.5 Pretraga

Niz omogućava pristup elementu po indeksu, u vremenu $O(1)$, jer su elementi smešteni na uzastopnim adresama, pa se adresa i -tog elementa izračuna iz adrese početka. Čvorovi liste nisu na uzastopnim adresama, pa takav pristup u listi ne postoji: klasa *Lista* nema operaciju koja vraća i -ti element.

Kod liste elemente možemo pretraživati po vrednosti. Pretraga se obavlja obilaskom, čvor po čvor:

```
template <typename T>
Cvor<T>* Lista<T>::pronadji(const T& x) const
{
    for (Cvor<T>* p = pocetak_; p != nullptr; p = p->sledeci)
        if (p->podatak == x)
            return p;
    return nullptr;
}
```

Funkcija vraća pokazivač na prvi čvor sa podatkom x , ili *nullptr* ako takvog čvora nema. U najgorem slučaju obilazi se cela lista, pa je složenost $O(n)$. Sortiranost liste ne ubrzava pretragu: binarna pretraga zahteva pristup srednjem elementu u vremenu $O(1)$, a lista takav pristup ne nudi.

Funkcija *pronadji* nije dovoljna za brisanje elementa sa zadatom vrednošću. Ona vraća čvor koji treba obrisati, a za brisanje nam je potreban njegov *prethodnik*. Zato pri obilasku ne ispituje tekući čvor, već njegovog sledbenika. Kada je sledbenik traženi čvor, pokazivač stoji tačno tamo gde treba.

```

template <typename T>
bool Lista<T>::obrisi(const T& x)
{
    if (prazna())
        return false;
    if (pocetak_ -> podatak == x) {
        obrisi_sa_pocetka();
        return true;
    }

    Cvor<T>* prethodni = pocetak_;
    while (prethodni -> sledeci != nullptr && prethodni -> sledeci -> podatak != x)
        prethodni = prethodni -> sledeci;

    if (prethodni -> sledeci == nullptr)
        return false;
    obrisi_iza(prethodni);
    return true;
}

```

Funkcija vraća *true* ako je element obrisan, a *false* ako ga u listi nema. Samo brisanje je složenosti $O(1)$, ali mu prethodi pretraga, pa je ukupna složenost $O(n)$. Prvi element se obrađuje posebno, jer on nema prethodnika. Ovakav poseban slučaj za početak liste videli smo i kod umetanja, a u odeljku 4.9 pokazujemo kako se on uklanja.

4.6 Sortirana lista i učešljavanje

Ista tehnika, obilazak sa pogledom na sledbenika, služi i za održavanje sortirane liste. Novi element umećemo iza poslednjeg čvora čiji je podatak manji od njega.

```

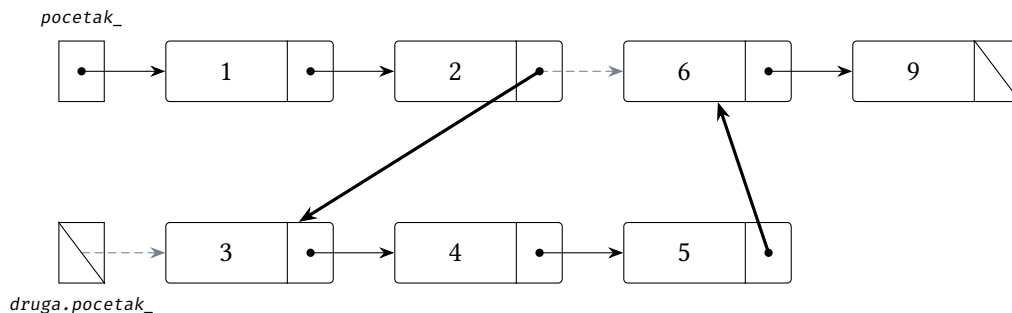
template <typename T>
void Lista<T>::umetni_sortirano(const T& x)
{
    if (prazna() || x <= pocetak_ -> podatak) {
        dodaj_na_pocetak(x);
        return;
    }

    Cvor<T>* prethodni = pocetak_;
    while (prethodni -> sledeci != nullptr && prethodni -> sledeci -> podatak < x)
        prethodni = prethodni -> sledeci;
    umetni_iza(prethodni, x);
}

```

Ako se elementi dodaju samo ovom funkcijom, lista je sve vreme sortirana. Jedno umetanje je složenosti $O(n)$, pa umetanje n elemenata zahteva $O(n^2)$ koraka. To je sortiranje umetanjem, izvedeno nad listom.

Lista je posebno pogodna za *učešljavanje* (eng. *merge*): spajanje dve sortirane liste u jednu sortiranu. Kod nizova je za rezultat potreban nov niz, u koji se elementi kopiraju. Kod lista se ništa ne kopira i ništa ne alocira, već se postojeći čvorovi samo drugačije povežu (slika 4.5). Postupak je isti kao kod nizova: poredimo prve preostale čvorove dve liste, manji od njih nadovezujemo na kraj rezultata i u njegovoj listi prelazimo na sledeći čvor. Kada u jednoj listi više nema čvorova, preostali deo druge liste nadovezuje se na rezultat odjednom, jednom izmenom pokazivača.



Slika 4.5: Učešljavanje dve sortirane liste. Čvorovi ostaju na svojim mestima u memoriji; menjaju se samo pokazivači.

Složenost učešljavanja je $O(n + m)$, gde su n i m dužine dve liste. Na njemu se zasniva sortiranje objedinjavanjem, koje listu sortira u vremenu $O(n \log n)$. Implementacija učešljavanja dostupna je u repozitorijumu uz knjigu, u folderu *kod/04-povezane-liste/*.

4.7 Poređenje liste i niza

Dinamički niz i povezana lista rešavaju isti zadatak – čuvaju više elemenata istog tipa u određenom redosledu – ali se razlikuju po načinu zauzimanja memorije, po složenosti operacija i po brzini na stvarnom računaru.

Niz zauzima jedan neprekidan blok memorije. Da bi rastao, mora da pronađe veći blok i da u njega prepíše sve elemente. Lista takvih zahteva nema: ona raste i smanjuje se čvor po čvor, a postojeći elementi se pri tome nikada ne premeštaju.

Neprekidnost, s druge strane, omogućava direktan pristup. Adresa i -tog elementa niza računa se iz adrese početka, pa je pristup složenosti $O(1)$, dok se u listi do i -tog elementa stiže praćenjem i pokazivača. Kod izmena je odnos obrnut: umetanje ili brisanje u nizu pomera elemente i složenosti je $O(n)$, a u listi je, na početku ili iza poznatog čvora, složenosti $O(1)$.

Operacija	Dinamički niz	Jednostruko povezana lista
pristup i -tom elementu	$O(1)$	$O(n)$
pretraga	$O(n)$	$O(n)$
umetanje i brisanje na početku	$O(n)$	$O(1)$
umetanje i brisanje iza poznate pozicije	$O(n)$	$O(1)$
dodavanje na kraj	amortizovano $O(1)$	$O(1)$ uz pokazivač na poslednji čvor
dodatna memorija	neiskorišćeni kapacitet	jedan pokazivač po čvoru

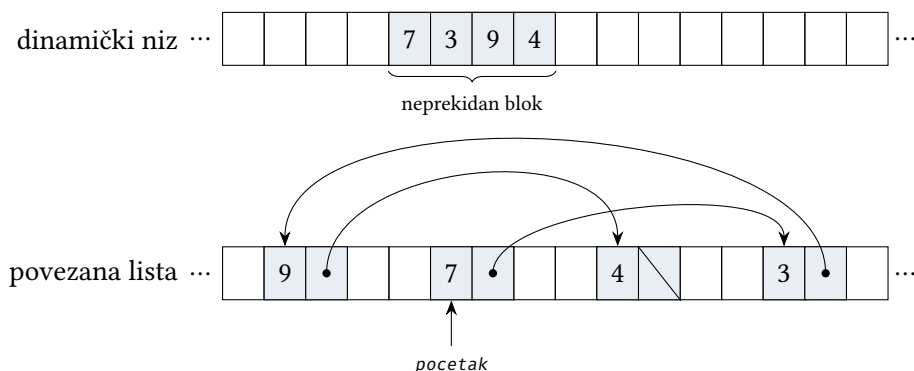
Prednosti liste plaćaju se memorijom. Uz svaki podatak čuva se i pokazivač, koji na 64-bitnim sistemima zauzima 8 bajtova. Niz od 10^6 elemenata tipa *int* zauzima oko 4 MB, a lista sa istim sadržajem oko 16 MB, jer jedan čvor zauzima 16 bajtova. Uz to, svaka alokacija čvora ima svoju cenu, pa mnogo malih alokacija usporava program.

4.7.1 Keš lokalnost

Obilazak niza i obilazak liste imaju istu složenost, $O(n)$. Na stvarnom računaru obilazak niza je ipak приметно brži, često i nekoliko puta. Razlog nije u broju koraka, nego u tome koliko traje jedan korak.

Pristup radnoj memoriji je za procesor spor. Zato procesor ima *keš memoriju* (eng. *cache*): malu i veoma brzu memoriju u kojoj čuva kopije podataka koje je nedavno koristio. Ako je traženi podatak u kešu, pristup je brz. Ako nije, podatak se mora dovući iz radne memorije, što može trajati i stotinu puta duže. Takav slučaj nazivamo *promašaj keša* (eng. *cache miss*).

Podaci se u keš ne prenose bajt po bajt, već u blokovima uzastopnih bajtova, koji najčešće imaju 64 bajta. Kada program pročita jedan element niza tipa *int*, u keš zajedno sa njim stiže i petnaest njegovih suseda, pa naredni pristupi ne koštaju gotovo ništa. Procesor uz to prepoznaje da program čita memoriju redom, pa naredne blokove dovlači unapred. Svojsvo programa da uzastopno pristupa bliskim adresama nazivamo *keš lokalnost* (eng. *cache locality*).



Slika 4.6: Isti elementi u memoriji. Dinamički niz (gore) zauzima jedan neprekidan blok, dok su čvorovi povezane liste (dole) rasuti, pa redosled elemenata određuju pokazivači, a ne redosled adresa.

Lista tu pogodnost nema (slika 4.6). Čvorovi su rasuti po memoriji, pa svaki prelazak na sledeći čvor može biti promašaj keša. Procesor ne može ni da dovlači podatke unapred, jer adresu sledećeg čvora saznaje tek kada pročita tekući.

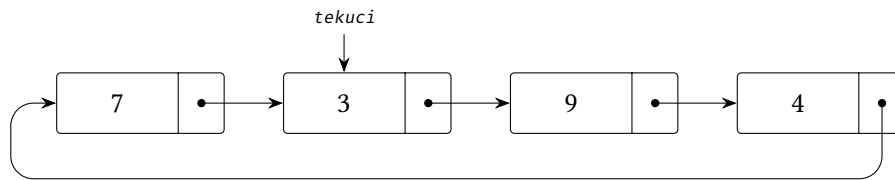
Posledica je važna za izbor strukture. Umetanje u sredinu niza pomera elemente, ali ih pomera redom, kroz susedne adrese, što procesor radi veoma brzo. Umetanje u listu je složenosti $O(1)$ tek kada je mesto poznato, a do mesta se stiže obilaskom, u kome je svaki korak spor. Zato je kod malih i srednjih kolekcija niz često brži od liste i u operacijama u kojima lista ima bolju složenost. Složenost govori kako cena raste sa brojem elemenata; ona ne govori koliko košta jedan korak.

Veličina čvora od 16 bajtova posledica je *poravnanja* (eng. *alignment*). Procesor najbrže čita podatak čija je adresa deljiva njegovom veličinom, pa pokazivač od 8 bajtova treba da počne na adresi deljivoj sa 8. Prevodilac zato između članova strukture ostavlja neiskorišćene bajtove: u čvoru sa podatkom tipa *int* iza 4 bajta podatka slede 4 prazna bajta, pa tek onda pokazivač. Stvarnu veličinu strukture daje operator *sizeof*, i ona može biti veća od zbira veličina njenih članova.

4.8 Kružna lista

Neke podatke prirodno obilazimo u krug: posle poslednjeg ponovo dolazi prvi. Tako se smenjuju igrači u igri na poteze, a tako i operativni sistem redom dodeljuje procesor programima koji čekaju.

Kružna lista (eng. *circular list*) je povezana lista u kojoj poslednji čvor ne pokazuje na *nullptr*, već na prvi čvor (slika 4.7). Čvorovi su isti kao kod jednostruko povezane liste; razlikuje se samo jedan pokazivač.



Slika 4.7: Jednostruko povezana kružna lista. Poslednji čvor pokazuje na prvi.

U kružnoj listi nema istaknutog početka ni kraja. Od svakog čvora može se stići do svakog drugog, pa je dovoljno čuvati pokazivač na bilo koji od njih, na primer na čvor koji je trenutno „na redu“. Obilazak se razlikuje od onog iz odeljka 4.3, jer nijedan pokazivač nije *nullptr*. Stajemo kada se vratimo u čvor iz kog smo krenuli:

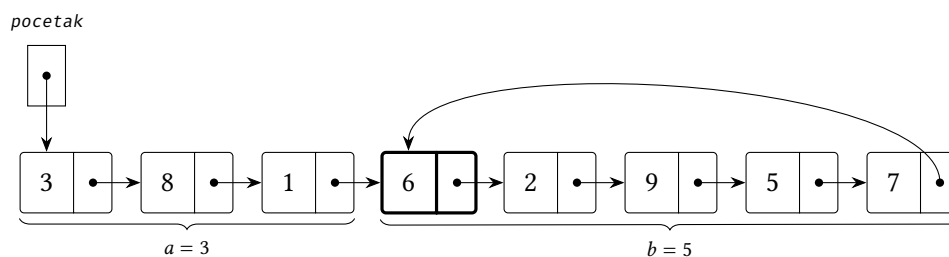
```
Cvor<T>* p = tekuci;
do {
    std::cout << p->podatak << ' ';
    p = p->sledeci;
} while (p != tekuci);
```

Petlja *do-while* je ovde neophodna. Na početku je *p* jednako *tekuci*, pa bi se petlja *while* sa istim uslovom završila pre prvog koraka. Prikazani kod pretpostavlja da lista nije prazna.

Tipičan zadatak za kružnu listu je sledeći: *n* plesača stoji u krugu, svaki *m*-ti izlazi, a brojanje se nastavlja od sledećeg, sve dok ne ostane samo jedan. Brojanje je pomeranje pokazivača, a izlazak plesača brisanje čvora u vremenu $O(1)$. Rešenje je dostupno u repozitorijumu uz knjigu, u folderu *kod/04-povezane-liste/*.

4.8.1 Otkrivanje ciklusa

Kružna lista je krug po nameri. Krug, međutim, može nastati i greškom. Dovoljno je da jedna pogrešna dodela usmeri pokazivač *sledeci* na neki raniji čvor, i obična lista dobija *ciklus* (slika 4.8). Invarijanta iz odeljka 4.2 tada više ne važi, a obilazak takve liste se nikada ne završava.

Slika 4.8: Lista sa ciklusom. Do prvog čvora ciklusa vodi *a* koraka, a ciklus ima *b* čvorova; ovde je $a = 3$ i $b = 5$.

Kako proveriti da li lista ima ciklus? Mogli bismo da pamtimo adrese svih posećenih čvorova i da u svakom koraku proverimo da li smo u tekućem već bili. Za to je potrebna dodatna memorija srazmerna dužini liste. *Floydov algoritam* rešava isti zadatak sa samo dva pokazivača.

Oba pokazivača kreću od početka liste. Prvi, *spori*, u svakom koraku prelazi jedan čvor, a drugi, *brzi*, dva čvora. Ako lista nema ciklus, brzi pokazivač stiže do *nullptr* i odgovor je poznat. Ako ciklus postoji, oba pokazivača pre ili kasnije ulaze u njega i više ga ne napuštaju. Tada brzi sustiže

sporog, kao što brži trkač na kružnoj stazi sustiže sporijeg: u svakom koraku razmak između njih smanjuje se za jedan, pa se moraju naći u istom čvoru.

Isti postupak, uz jedan dodatak, pronalazi i čvor u kom ciklus počinje. Posle susreta spori pokazivač vraćamo na početak liste, a zatim oba pomeramo po jedan čvor. Čvor u kom se ponovo sretnu je prvi čvor ciklusa.

```
template <typename T>
Cvor<T>* pocetak_ciklusa(Cvor<T>* pocetak)
{
    Cvor<T>* spori = pocetak;
    Cvor<T>* brzi = pocetak;

    while (brzi != nullptr && brzi->sledeci != nullptr) {
        spori = spori->sledeci;
        brzi = brzi->sledeci->sledeci;

        if (spori == brzi) {
            spori = pocetak;
            while (spori != brzi) {
                spori = spori->sledeci;
                brzi = brzi->sledeci;
            }
            return spori;
        }
    }
    return nullptr;
}
```

Funkcija vraća pokazivač na prvi čvor ciklusa, ili *nullptr* ako ciklusa nema. Pokazivači se porede po adresi, a ne po podatku, jer dva različita čvora mogu sadržati isti podatak.

Ostaje da se dokaže da postupak radi ispravno. Neka od početka liste do prvog čvora ciklusa ima a koraka i neka ciklus ima b čvorova. Numerišimo čvorove redom od početka liste brojevima $0, 1, \dots, a+b-1$. Čvorovi $0, \dots, a-1$ čine linearni deo liste, čvor a je prvi čvor ciklusa, a iz poslednjeg čvora, $a+b-1$, ponovo se prelazi u čvor a . Pokazivač koji je od početka liste napravio s koraka nalazi se u čvoru

$$c(s) = \begin{cases} s, & s < a, \\ a + (s - a) \bmod b, & s \geq a. \end{cases}$$

Za listu sa slike 4.8 je $a = 3$ i $b = 5$. Posle t koraka spori pokazivač je u čvoru $c(t)$, a brzi u čvoru $c(2t)$:

t	0	1	2	3	4	5
spori, $c(t)$	0	1	2	3	4	5
brzi, $c(2t)$	0	2	4	6	3	5

Pokazivači se sreću posle $t_0 = 5$ koraka, u čvoru 5. Zatim se spori vraća na početak, a brzi nastavlja iz čvora 5, i oba prave po jedan korak:

k	0	1	2	3
vraćeni, $c(k)$	0	1	2	3
drugi, $c(t_0 + k)$	5	6	7	3

Posle $a = 3$ koraka oba su u čvoru 3, prvom čvoru ciklusa. Pokažimo da je tako uvek.

Teorema 4.8.1. *Ako lista ima ciklus, spori i brzi pokazivač se sreću posle t_0 koraka, gde je $t_0 \geq a$ i t_0 je deljivo sa b .*

Dokaz. Pokazivači se posle $t \geq 1$ koraka sreću ako i samo ako je $c(t) = c(2t)$.

Ako je $t \geq a$, onda je

$$c(t) = c(2t) \iff (t - a) \bmod b = (2t - a) \bmod b \iff b \mid t.$$

Broj $t \geq a$ deljiv sa b postoji, na primer $t = (a + 1)b$, pa se pokazivači sreću, a prvi put u trenutku t_0 , najmanjem takvom broju. $\square$

Teorema 4.8.2. *Ako se posle susreta spori pokazivač vrati na početak liste, a zatim se oba pokazivača pomeraju po jedan čvor, oni se prvi put sretnu posle a koraka, u čvoru a , prvom čvoru ciklusa.*

Dokaz. Posle k novih koraka vraćeni pokazivač je u čvoru $c(k)$, a drugi u čvoru $c(t_0 + k)$. Za $k = a$ je $c(a) = a$ i, pošto je t_0 deljivo sa b ,

$$c(t_0 + a) = a + t_0 \bmod b = a.$$

Oba pokazivača su, dakle, u čvoru a . $\square$

Ostaje složenost. Lista ima $n = a + b$ čvorova. Među brojevima $a, a + 1, \dots, a + b - 1$, kojih ima b , tačno jedan je deljiv sa b , pa je $t_0 \leq a + b - 1 < n$. Prvi deo algoritma, dakle, traje manje od n koraka, a drugi $a < n$ koraka, pa je ukupna složenost $O(n)$. Dodatna memorija je $O(1)$.

4.9 Dvostruko povezana lista

Jednostruko povezana lista ima dva ograničenja koja smo već sreli. Kroz nju se može kretati samo unapred, a čvor se ne može obrisati ako se ne zna njegov prethodnik. Oba potiču iz istog uzroka: čvor ne zna ko mu prethodi.

Dvostruko povezana lista (eng. *doubly linked list*) to ispravlja tako što svaki čvor čuva dva pokazivača: na sledeći i na prethodni čvor. Iz svakog čvora može se zato preći i na sledbenika i na prethodnika, pa se umetanje i brisanje izvode u vremenu $O(1)$ na svakom mestu u listi, čim je poznat pokazivač na čvor. Cena je memorija: čvor ima jedan pokazivač više, pa čvor sa podatkom tipa *int* zauzima 24 bajta umesto 16. Uz to, pri svakoj izmeni treba ispravno postaviti dvostruko više pokazivača.

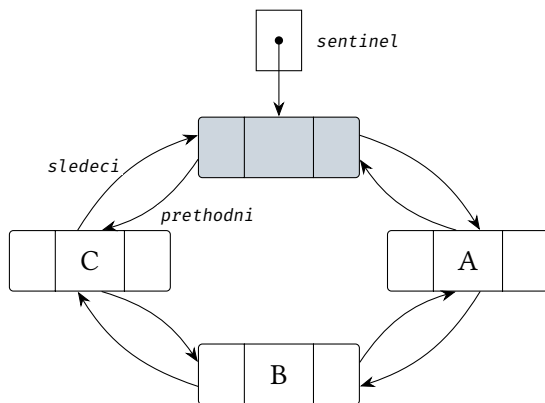
4.9.1 Sentinel-čvor

Operacije nad jednostruko povezanom listom razlikovale su dva slučaja: izmenu na početku liste, gde se menja *pocetak_*, i izmenu iza nekog čvora, gde se menja njegovo polje *sledeci*. Kod dvostruke liste takvih slučajeva bilo bi još više, jer je poseban i početak i kraj, a svaki čvor ima dva pokazivača. Svaki poseban slučaj znači jedno grananje više u kodu i jedno mesto više na kom se može pogrešiti.

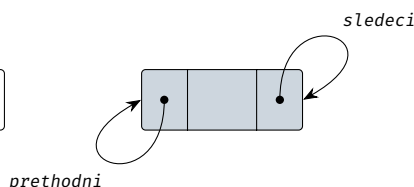
Posebni slučajevi nestaju ako se u listu doda *sentinel-čvor* (eng. *sentinel node*). To je pomoćni čvor koji ne sadrži nijedan element liste, a postoji uvek, i kada je lista prazna. Sentinel-čvor stavljamo

između poslednjeg i prvog elementa: poslednji element pokazuje na njega kao na svog sledbenika, a prvi kao na svog prethodnika. Dobijamo kružnu listu iz prethodnog odeljka, ovog puta dvostruko povezanu, u kojoj je sentinel-čvor mesto na kom se kraj liste spaja sa početkom (slika 4.9). U praznoj listi sentinel-čvor pokazuje sam na sebe, u oba smeru.

(a) lista sa tri elementa



(b) prazna lista



Slika 4.9: Dvostruko povezana lista sa sentinel-čvorom: (a) lista sa tri elementa; (b) prazna lista. Sentinel-čvor je osenčen.

Zahvaljujući sentinel-čvoru, svaki čvor ima i prethodnika i sledbenika. Nijedan pokazivač u listi nije *nullptr*, pa nema ni provera za *nullptr*. To se vidi i u invarijanti, koja je jednostavnija nego kod jednostruke liste.

Invarijanta dvostruko povezane liste. Za svaki čvor *c*, uključujući i sentinel-čvor, važi $c \rightarrow \text{sledeci} \rightarrow \text{prethodni} == c$ i $c \rightarrow \text{prethodni} \rightarrow \text{sledeci} == c$. Između dva prolaska kroz sentinel-čvor nalazi se tačno *velicina_* čvorova.

Drugim rečima: ako iz čvora odemo korak napred pa korak nazad, vraćamo se u isti čvor.

4.9.2 Klasa *DvostrukaLista*

```
template <typename T>
class DvostrukaLista {
public:
    struct Cvor {
        T podatak;
        Cvor* prethodni;
        Cvor* sledeci;

        Cvor() : podatak(), prethodni(this), sledeci(this) {}
        Cvor(const T& p, Cvor* pre, Cvor* sl)
            : podatak(p), prethodni(pre), sledeci(sl) {}
    };

    DvostrukaLista() : sentinel(new Cvor()), velicina_(0) {}

    ~DvostrukaLista()
    {
```

```

    while (!prazna())
        obrisi(prvi());
    delete sentinel;
}

DvostrukaLista(const DvostrukaLista&) = delete;
DvostrukaLista& operator=(const DvostrukaLista&) = delete;

bool prazna() const { return sentinel->sledeci == sentinel; }
std::size_t velicina() const { return velicina_; }

Cvor* prvi() const { return sentinel->sledeci; }
Cvor* poslednji() const { return sentinel->prethodni; }
Cvor* kraj() const { return sentinel; }

void umetni_ispred(Cvor* c, const T& x);
void obrisi(Cvor* c);

void dodaj_na_pocetak(const T& x) { umetni_ispred(prvi(), x); }
void dodaj_na_kraj(const T& x) { umetni_ispred(kraj(), x); }

private:
    Cvor* sentinel;
    std::size_t velicina_;
};

```

Čvor je ovde definisan unutar klase. Tip čvora je deo građe liste i van nje nema drugu namenu, pa mu je tu prirodno mesto. Uz to, svaka lista tako može imati sopstveni tip *Cvor*, bez sudara imena: pun naziv ovog tipa je *DvostrukaLista<T>::Cvor*.

Čvor ima dva konstruktora. Prvi, bez parametara, pravi sentinel-čvor: oba pokazivača postavlja na *this*, tj. na sam taj čvor, što je upravo prazna lista sa slike 4.9. Drugi pravi običan čvor sa zadatim podatkom i zadatim susedima.²

Klasa čuva samo pokazivač na sentinel-čvor. Do oba kraja liste stiže se iz njega jednim korakom: *prvi()* je sledbenik sentinel-čvora, a *poslednji()* njegov prethodnik. Funkcija *kraj()* vraća sam sentinel-čvor. On označava mesto iza poslednjeg elementa, pa služi kao granica pri obilasku. Lista se obilazi unapred ili unazad, a u oba smera staje se kod sentinel-čvora:

```

for (auto p = lista.prvi(); p != lista.kraj(); p = p->sledeci)
    std::cout << p->podatak << ' ';

for (auto p = lista.poslednji(); p != lista.kraj(); p = p->prethodni)
    std::cout << p->podatak << ' ';

```

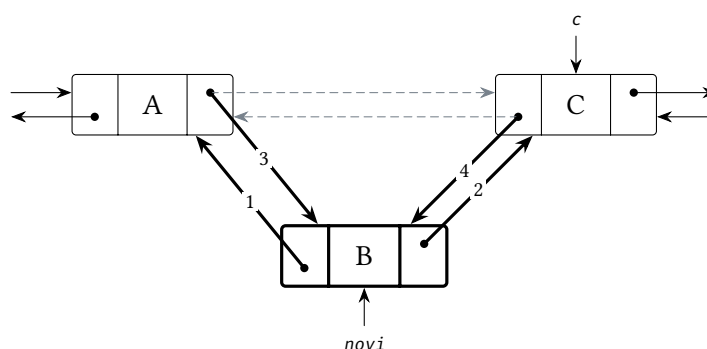
Ključna reč *auto* prepušta prevodiocu da sam odredi tip promenljive, na osnovu vrednosti kojom je inicijalizujemo. Ovde je to *DvostrukaLista<T>::Cvor**, pa dugačko ime tipa ne moramo da pišemo.

²I sentinel-čvor ima polje *podatak*, iako se ono nikada ne koristi. Zato tip *T* mora imati podrazumevani konstruktor. Isto ograničenje ima i klasa *DinamickiNiz* (odjeljak 3.2).

4.9.3 Umetanje i brisanje

Novi čvor umećemo ispred zadatog čvora *c*. On dolazi između čvora *c* i njegovog dosadašnjeg prethodnika, pa treba postaviti četiri pokazivača: dva u novom čvoru i po jedan u svakom susedu (slika 4.10).

```
template <typename T>
void DvostrukaLista<T>::umetni_ispred(Cvor* c, const T& x)
{
    Cvor* novi = new Cvor(x, c->prethodni, c);
    c->prethodni->sledeci = novi;
    c->prethodni = novi;
    ++velicina_;
}
```



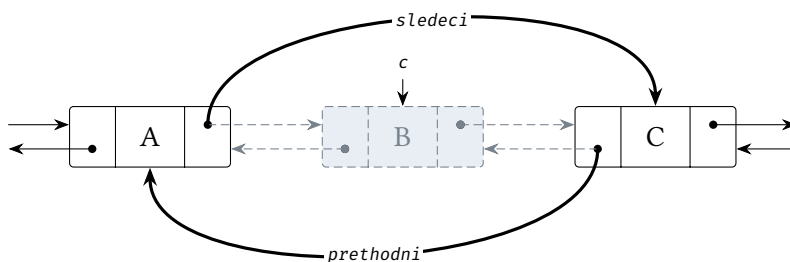
Slika 4.10: Umetanje elementa ispred čvora *c*. Brojevi označavaju redosled kojim se postavljaju pokazivači.

Konstruktor postavlja oba pokazivača novog čvora. Zatim dosadašnji prethodnik čvora *c* dobija novog sledbenika, a čvor *c* novog prethodnika. Redosled poslednje dve dodele je važan: u prvoj se još koristi stara vrednost *c->prethodni*, pa se ona sme izmeniti tek u drugoj dodeli.

U funkciji nema nijednog grananja. Ona radi isto na početku liste, u sredini i na kraju, pa i kada je lista prazna. Zato se dodavanje na početak i dodavanje na kraj svode na po jedan poziv: umetanje ispred prvog elementa, odnosno umetanje ispred sentinel-čvora. Ovo drugo možda deluje neobično, ali prethodnik sentinel-čvora je poslednji element, pa novi čvor dolazi tačno iza njega.

Da bismo obrisali čvor *c*, njegove susede povezujemo međusobno, a zatim čvor oslobađamo (slika 4.11).

```
template <typename T>
void DvostrukaLista<T>::obrisi(Cvor* c)
{
    assert(c != sentinel);
    c->prethodni->sledeci = c->sledeci;
    c->sledeci->prethodni = c->prethodni;
    delete c;
    --velicina_;
}
```



Slika 4.11: Brisanje čvora *c*: susedi se povezuju međusobno, a čvor se oslobađa.

Za razliku od jednostruke liste, ovde je dovoljan pokazivač na čvor koji se briše, jer do prethodnika stižemo preko polja *prethodni*. Jedini čvor koji se ne sme obrisati je sentinel-čvor, i na to upozorava *assert*. Njega oslobađa tek destruktor, pošto obriše sve elemente.

Umetanje i brisanje su složenosti $O(1)$ i nemaju posebnih slučajeva. Vredi ih uporediti sa funkcijom *obrisi* iz odeljka 4.5, koja je posebno obrađivala praznu listu, prvi element i ostale elemente. Sentinel-čvor je sve te slučajeve sveo na jedan, po cenu jednog dodatnog čvora. Potpuna implementacija dvostruko povezane liste dostupna je u repozitorijumu uz knjigu, u folderu *kod/04-povezane-liste/*.

4.10 Liste u standardnoj biblioteci

4.10.1 Iteratori

U odeljku 2.3 niz smo obišli pokazivačem: krenuli smo od prvog elementa, operatorom *++* prelazili na sledeći i stali kod mesta iza poslednjeg. Listu obilazimo po istom obrascu, samo što umesto *++p* pišemo *p = p->sledeci*. Iterator je način da se taj obrazac zapiše isto za svaku strukturu.

Obilazak pokazivačem na čvor ima dve mane. Prvo, korisnik mora da zna kako je lista građena: da postoji čvor i da se njegova polja zovu *podatak* i *sledeci*. Drugo, preko tog pokazivača korisnik može i da pokvari listu. Dovoljna je naredba *p->sledeci = nullptr* i invarijanta više ne važi.

Kontejneri standardne biblioteke zato ne otkrivaju svoje čvorove. Umesto pokazivača na čvor korisnik dobija *iterator* (eng. *iterator*): mali objekat koji označava jednu poziciju u kontejneru. Iterator se koristi slično kao pokazivač, ali dozvoljava samo ono što je bezbedno: pristup elementu i prelazak na susednu poziciju.

Radnja	Pokazivač na čvor	Iterator
prvi element	<i>p = lista.prvi()</i>	<i>it = lista.begin()</i>
pristup elementu	<i>p->podatak</i>	<i>*it</i>
prelazak na sledeći	<i>p = p->sledeci</i>	<i>++it</i>
prelazak na prethodni	<i>p = p->prethodni</i>	<i>--it</i>
provera kraja	<i>p != lista.kraj()</i>	<i>it != lista.end()</i>

Iterator liste u sebi čuva upravo pokazivač na čvor, a operator *++* izvršava naredbu *p = p->sledeci*. Razlika je u tome što korisnik to ne vidi i ne može da promeni. Kao i funkcija *kraj()* u klasi *DvostrukaLista*, *end()* ne označava poslednji element, već poziciju iza njega.

Iteratori imaju još jednu prednost. Sve kontejnere standardne biblioteke obilazimo na isti način, iako su iznutra potpuno različiti:

```
for (auto it = kontejner.begin(); it != kontejner.end(); ++it)
    std::cout << *it << ' ';
```

Kod niza operator `++` prelazi na sledeću adresu u memoriji, a kod liste prati pokazivač, ali petlja izgleda isto. Kraći zapis iste petlje je `for (const auto& x : kontejner)`.

Iterator nudi samo one operacije koje kontejner može efikasno da izvrši. Iterator kontejnera `std::vector` podržava i `it + 5`, jer se u nizu preskače u vremenu $O(1)$. Iterator dvostruko povezane liste podržava samo `++` i `--`, a iterator jednostruko povezane samo `++`.

4.10.2 Kontejneri `std::forward_list` i `std::list`

Jednostruko povezanoj listi odgovara kontejner `std::forward_list<T>`, iz zaglavlja `<forward_list>`, a dvostruko povezanoj `std::list<T>`, iz zaglavlja `<list>`. Sve operacije iz sledeće tabele su složenosti $O(1)$.

<code>std::forward_list</code>	<code>std::list</code>	Opis
<code>push_front(x)</code>	<code>push_front(x)</code> , <code>push_back(x)</code>	umetanje na početak, odnosno na kraj
<code>pop_front()</code>	<code>pop_front()</code> , <code>pop_back()</code>	brisanje sa početka, odnosno sa kraja
<code>insert_after(it, x)</code>	<code>insert(it, x)</code>	umetanje iza, odnosno ispred pozicije <code>it</code>
<code>erase_after(it)</code>	<code>erase(it)</code>	brisanje iza pozicije <code>it</code> , odnosno na njoj

Operacije kontejnera `std::forward_list` odgovaraju funkcijama klase `Lista`, a `insert` i `erase` kontejnera `std::list` funkcijama `umetni_ispred` i `obrisi` klase `DvostrukaLista`; razlika je u tome što poziciju zadaje iterator. Uobičajene implementacije kontejnera `std::list` građene su kao i prikazana klasa: kao kružna lista sa sentinel-čvorom, na čiju poziciju pokazuje `end()`.

Za razliku od ostalih kontejnera, `std::forward_list` nema funkciju `size()`. Nijedna od dve liste nema ni operator `[]`: pristup i -tom elementu bio bi složenosti $O(n)$, pa ga biblioteka namerno ne nudi.

4.10.3 Primer: linijski editor

Posmatrajmo jednu liniju teksta u editoru. Kursor se pomera ulevo i udesno, karakter se umeće na mestu kursora, a karakter se može brisati levo od kursora ili desno od njega. Ako bi linija bila niz, svako umetanje i brisanje u sredini pomeralo bi sve karaktere iza kursora. U listi su to operacije složenosti $O(1)$, a pošto se kursor kreće u oba smera, lista mora biti dvostruko povezana.

Kursor se može prirodno predstaviti iteratorom. On stoji između dva karaktera, pa neka bude dogovor da iterator pokazuje na karakter desno od kursora. Kada je kursor na kraju linije, iterator je jednak `end()`. Sledeća funkcija izvršava niz komandi: `iX` umeće karakter `X`, `< i >` pomeraju kursor, `b` briše karakter levo, a `d` karakter desno od kursora.

```
std::string uredi(const std::string& komande)
{
    std::list<char> linija;
    auto kursor = linija.end();
```

```

for (std::size_t i = 0; i < komande.size(); ++i) {
    char k = komande[i];
    if (k == 'i') {
        ++i;
        linija.insert(kursor, komande[i]);
    } else if (k == '<') {
        if (kursor != linija.begin())
            --kursor;
    } else if (k == '>') {
        if (kursor != linija.end())
            ++kursor;
    } else if (k == 'b') {
        if (kursor != linija.begin())
            linija.erase(std::prev(kursor));
    } else if (k == 'd') {
        if (kursor != linija.end())
            kursor = linija.erase(kursor);
    }
}
return std::string(linija.begin(), linija.end());
}

```

Funkcija `std::prev`, iz zaglavlja `<iterator>`, vraća iterator na prethodnu poziciju. Umetanje ne pomera kursor, jer `insert` umeće ispred pozicije na koju iterator pokazuje. Kod brisanja udesno briše se upravo element na koji kursor pokazuje, pa bi iterator posle toga bio neispravan. Zato koristimo vrednost koju vraća `erase`: iterator na element iza obrisanog. Za komande `iAiB<biC` funkcija vraća `CB`: posle unosa slova `A` i `B` kursor se vraća jedno mesto ulevo, briše se `A`, a zatim se na to mesto umeće `C`.

4.11 Kada lista jeste, a kada nije dobar izbor

Lista je dobar izbor kada su umetanja i brisanja na poznatim pozicijama glavne operacije, a pristup po indeksu nije potreban. Takav je slučaj kod heš-tabele sa ulančavanjem (glava ??), a lista je i prirodna osnova za stek i red (glave ?? i ??), kod kojih se sve izmene dešavaju na krajevima. Lista je pravi izbor i kada se delovi kolekcije često premeštaju iz jedne kolekcije u drugu.

Lista nije dobar izbor kada se elementima pristupa po indeksu, kada se kolekcija pretražuje binarnom pretragom ili kada se velika kolekcija često obilazi cela. U prva dva slučaja lista ima lošiju složenost, a u trećem istu složenost, ali sporiji korak, zbog keš memorije. Lista je nepovoljna i kada su elementi mali, jer tada pokazivači zauzimaju više memorije od samih podataka. Lista se teško deli na delove koji bi se izvršavali istovremeno, jer se do sredine liste stiže samo obilaskom, pa nije moguće efikasno paralelizovati rad sa listom.

5

Stek

5.1 ADT steka

5.2 Operacije: *push*, *pop*, *top*, *empty*

5.3 Implementacija pomoću niza ili vektora

5.4 Implementacija pomoću liste kao alternativa

5.5 Primene

5.6 Veza sa *std::stack*

1.3.9 Prenos argumenata preko steka

6

Red i dek

6.1 ADT reda

6.2 Operacije nad redom

6.3 Implementacija kružnim baferom

6.4 Implementacija pomoću povezane liste

6.5 Red pomoću dva steka kao primer amortizovane analize

6.6 Primene

6.7 Dek kao red sa dva kraja

6.8 Osnovna ideja implementacije deka

6.9 Veza sa *std::queue* i *std::deque*

7

Hip i red sa prioritetom

7.1 Ideja prioriteta

7.2 Binarni hip

7.3 Implicitna reprezentacija u nizu

7.4 Operacije

7.5 Formiranje hipa

7.6 Hipsort

7.7 Primene reda sa prioritetom

7.8 Veza sa *std::priority_queue*

To je posebna vrsta reda sa prioritetom, u kojoj se dva reda mogu efikasno spojiti u jedan (binomni, levičarski i slični hipovi), i ona se zaista gradi od povezanih čvorova.? ovo je zanimljivo??

8

Binarna stabla

8.1 Rekurzivna struktura stabla

8.2 Čvor, koren, list, visina

8.3 Obilasci stabla

8.4 Rekurzivni algoritmi nad stablima

8.5 Memorijska reprezentacija stabla

9

Binarna stabla pretrage

9.1 Uređenje elemenata

9.2 Invarijanta BST-a

9.3 Pretraga

9.4 Umetanje

9.5 Brisanje

9.6 Složenost u zavisnosti od visine

9.7 Problem degenerisanog stabla

10

Balansirana stabla

10.1 Motivacija za balansiranje

10.2 Visina i složenost

10.3 AVL stablo ili crveno-crno stablo kao glavni primer

10.4 Rotacije

10.5 Umetanje

10.6 Brisanje

10.7 Veza sa `std::map` i `std::set`

11

Heš-tabele

11.1 Asocijativni pristup podacima

11.2 Ključ i vrednost

11.3 Heš-funkcija

11.4 Kolizije

11.5 Odvojeno ulančavanje

11.6 Otvoreno adresiranje

11.7 Faktor opterećenja

11.8 Proširivanje i rehash

11.9 Prosečna i najgora složenost

11.10 Veza sa `std::unordered_map` i `std::unordered_set`

12

Dodatne i napredne strukture

12.1 Skip-lista

12.2 B-stablo (B+ stablo)

A

Izbor strukture podataka u praksi

A.1 Kada koristiti vektor

A.2 Kada koristiti listu

A.3 Kada koristiti stek

A.4 Kada koristiti red

A.5 Kada koristiti hip

A.6 Kada koristiti stablo

A.7 Kada koristiti heš-tabelu

A.8 Primeri

B

Kratak pregled C-a

B.1 Osnovna sintaksa (C i C++ razlike)

B.2 C nizovi

B.3 C niske

B.4 *malloc* i *free*

B.5 *printf* i *scanf*

B.6 C datoteke

B.6.1 Treci nivo

Korak